UNIVERSIDAD NACIONAL DEL SUR
Departamento de Ciencias e Ingeniería de la Computación
Compiladores e Intérpretes — Segundo Cuatrimestre de 2026

Compilador de MiniJava

Etapa 2 — Analizador Sintáctico

Descendente Recursivo Predictivo

Agustín Paul
L.U. 147524

Informe de entrega

Transformación de la gramática a LL(1), paso a paso.
Las nueve extensiones por logro y su verificación.

16 de septiembre de 2026

Índice

1. Alcance **1**
1.1. Qué está y qué no está en este informe . . . 1
1.2. Qué chequea y qué no chequea un analizador sintáctico . . . 1

2. Notación **2**
2.1. El criterio LL(1) . . . 2
2.2. Dos defectos distintos que conviene no confundir . . . 2

3. La gramática original, comentada **3**
3.1. Estructura del programa . . . 3
3.2. Miembros . . . 3
3.3. Tipos . . . 4
3.4. Sentencias . . . 4
3.5. Expresiones . . . 5
3.6. Referencias y primarios . . . 6
3.7. Construcciones que faltan . . . 6

4. Diagnóstico **6**
4.1. Cuántos de esos dieciséis son ambigüedades . . . 7
4.2. Las dos ambigüedades . . . 8
4.3. Una tercera ambigüedad heredada, y una que agrega un logro . . . 8

5. Transformaciones **9**
5.1. Orden de trabajo y criterios . . . 9
5.2. Eliminación de recursión a izquierda . . . 10
5.2.1. R1 — `<ListaArgsFormales>` . . . 10
5.2.2. R2 — `<ExpresionCompuesta>` . . . 11
5.2.3. R3 — `<Referencia>`, y el conflicto que la transformación genera . . . 11
5.3. Factorización a izquierda . . . 12
5.3.1. F1 — `<Expresion>` . . . 12
5.3.2. F2 — `<Sentencia>`: resolver la ambigüedad . . . 12
5.3.3. F3 — `<If>`: factorizar, y después resolver el *dangling else* . . . 13
5.3.4. F4 — `<DimensionesConTamanio>` . . . 13
5.3.5. F5 — `<ListaExps>` . . . 13
5.3.6. F6 — `<Miembro>`: factorización profunda . . . 13
5.3.7. F7 — `<Primario>`: el prefijo **idMetVar** . . . 14
5.3.8. F8 — `<Primario>`: el prefijo **new**, a dos niveles . . . 15
5.4. Estado del núcleo . . . 15

6. Extensiones por logro **15**
6.1. La condición de atomicidad . . . 15
6.2. Visibilidad Mejorada E2 . . . 16
6.3. Atributos Inicializados y Variables locales Clásicas E2 . . . 17
6.4. Fors! E2 . . . 18
6.5. Operador Ternario E2 . . . 18
6.6. Operadores Posifijos E2 . . . 19

6.7. MiniLambdas E2 . . . 19
6.8. Inicializacion de Arreglos E2 . . . 22
6.9. Genericidad Avanzada E2 . . . 23
6.10. Floats (*carry-over* de la Etapa 1) . . . 23
6.11. Impacto sobre el analizador léxico . . . 24

7. Gramática final **24**

8. Cómo compilar y correr **32**

9. El analizador sintáctico **33**
9.1. La técnica . . . 33
9.2. Las clases . . . 33
9.3. Los tipos de error que detecta . . . 34
9.4. El formato del reporte . . . 35

10. Logros perseguidos **35**

11. Verificación **36**
11.1. Condición LL(1) . . . 36
11.2. Atomicidad de los logros . . . 37
11.3. Ambigüedad frente a conflicto de predicción . . . 37
11.4. Registro de ambigüedades . . . 37
11.5. Casos de aceptación y rechazo . . . 38
11.6. Validación contra los casos de la cátedra . . . 38
11.7. Tres reconocedores independientes de la misma gramática . . . 39
11.8. Equivalencia de lenguajes con la gramática de la cátedra . . . 39
11.9. El analizador cita exactamente la gramática . . . 40
11.10 Resumen . . . 40
11.11 Tabla de Primeros . . . 41

12. Decisiones tomadas con criterio propio **53**
12.1. Ambigüedades: qué árbol se elige . . . 53
12.2. Dónde la especificación admitía más de una lectura . . . 54
12.3. Qué verifica el analizador y qué queda para el semántico . . . 55
12.4. Decisiones de implementación . . . 55

13. Herramientas y uso de inteligencia artificial **56**
13.1. Herramientas desarrolladas . . . 56
13.2. Uso de inteligencia artificial . . . 56

1 Alcance

Este informe documenta la Etapa 2 del compilador de MiniJava: el **Analizador Sintáctico Descendente Recursivo Predictivo**. Su objeto central es la **gramática**: se parte de la que publica la cátedra, se la diagnostica, se la corrige a LL(1) paso a paso, se la extiende con los nueve logros perseguidos y se verifica el resultado.

El criterio de exposición es que **cada corrección quede localizada en un paso concreto**. Por eso las transformaciones no se presentan como un salto de la gramática vieja a la nueva, sino en cuatro pasos:

1. la **regla original**, tal cual el documento de la cátedra;
2. el **defecto** detectado, con los conjuntos de Primeros que se solapan;
3. la **transformación** aplicada, citando el esquema de la teoría que la justifica;
4. el **resultado** y la verificación de que los Primeros quedan disjuntos.

1.1 Qué está y qué no está en este informe

Está la gramática y su tratamiento, y está el analizador que sale de ella: cómo compilarlo y correrlo (sección 8), cómo está construido y qué errores detecta (sección 9), qué logros persigue (sección 10) y qué decisiones hubo que tomar donde la especificación admitía más de una lectura (sección 12).

1.2 Qué chequea y qué no chequea un analizador sintáctico

Conviene fijarlo desde el principio, porque delimita todo lo que sigue.

El analizador sintáctico verifica que la secuencia de tokens esté en un orden que la gramática pueda generar: la estructura de clases, interfaces, miembros, bloques y sentencias; que los delimitadores abran y cierren; que una expresión esté bien formada; que un método tenga cuerpo y un método de interfaz no.

No verifica nada que dependa del *significado* de los nombres. Todo lo siguiente atraviesa el análisis sintáctico sin objeción y corresponde al analizador semántico:

Pasa el análisis sintáctico	**Por qué es semántico**
`int x = "hola";`	compatibilidad de tipos
`noDeclarada = 5;`	declaración y alcance de nombres
`5 = x;`	que el lado izquierdo sea asignable
`class A extends UnaInterfaz {}`	que el padre sea realmente una clase
`m(1,2,3)` con `m` de un argumento	aridad contra la declaración
`int x; int x;`	declaraciones duplicadas
`return;` en un método `int`	compatibilidad del valor de retorno
una lambda que usa una variable de afuera	captura de variables

El último caso merece atención porque afecta a un logro. El enunciado de *MiniLambdas E2* prohíbe la captura de variables locales y de parámetros, pero eso **no es decidible**

sintácticamente: el analizador no sabe qué nombres están en alcance en cada punto. Lo único de ese logro que la gramática puede imponer es la *forma* — parámetros sin tipo, cuerpo de una única expresión, sin llaves — y así se hace. La restricción sobre la captura queda anotada como responsabilidad del analizador semántico.

2 Notación

Se usa la misma notación BNF-extendida del documento *Reglas de Sintaxis de MiniJava* de la cátedra, con la misma convención tipográfica, para que las reglas de este informe se puedan comparar renglón a renglón con las de aquél.

Elemento	Significado
terminal	símbolo terminal
`<Clase>`	símbolo no terminal (primera letra mayúscula)
ϵ	la cadena vacía
`<X> ::=` α	una producción, con α una secuencia de terminales y no terminales
`<X> ::=` α | β	abreviación de `<X> ::=` α y `<X> ::=` β
Primeros(β)	conjunto de terminales que pueden aparecer primeros en β

El no terminal inicial es `<Inicial>`.

2.1 El criterio LL(1)

Una gramática es LL(1) si, para todo no terminal A con producciones $A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$, se cumple para todo $i \neq j$:

$$\text{Primeros}(\beta_i) \cap \text{Primeros}(\beta_j) = \varnothing$$

Sólo con esa condición un único token de anticipación alcanza para elegir la producción correcta sin necesidad de *backtracking*. Cuando la condición no se cumple, la gramática debe adaptarse **sin cambiar el lenguaje que genera**, que es la restricción que gobierna todas las transformaciones de este informe.

2.2 Dos defectos distintos que conviene no confundir

Buena parte del trabajo consistió en separar dos cosas que se parecen y no son lo mismo.

- **Ambigüedad — el lenguaje está mal descripto**

  Existe una cadena con **dos árboles de derivación distintos**. No queda determinado qué significa el programa. No siempre se puede arreglar reescribiendo: a veces hay que *elegir* cuál de los dos significados vale.

- **Conflicto LL(1) — el analizador no puede predecir**

  Dos producciones del mismo no terminal comparten Primeros. La cadena sigue teniendo **un solo** árbol; lo que falla es que el analizador no puede saber cuál producción aplicar mirando un token. Se corrige factorizando o eliminando recursión, y el lenguaje generado **no cambia**.

Todo par de producciones que comparte prefijo izquierdo produce un conflicto LL(1); no todo par que comparte prefijo izquierdo es ambiguo. Si los dos lados derivan lenguajes disjuntos, no hay ninguna cadena con dos árboles.

Un criterio suficiente y barato para probar esa disyunción es comparar los **rangos de longitud derivable**: si no se solapan, los lenguajes son disjuntos y no puede haber ambigüedad. Se aplica en la sección 4 y decide varios de los casos.

3 La gramática original, comentada

Antes de tocar nada, un recorrido por la gramática de la cátedra explicando qué modela cada regla y **por qué está escrita así**. Varias decisiones de forma que parecen arbitrarias son en realidad restricciones del lenguaje impuestas por la gramática, y perderlas al transformar sería cambiar MiniJava.

3.1 Estructura del programa

`<Inicial>` ::= `<ListaClases>` **eof**

`<ListaClases>` ::= `<Clase>` `<ListaClases>` | `<Interfaz>` `<ListaClases>` | ϵ

El **eof** no es decoración: cumple la función del **$** de la teoría y obliga al analizador a consumir *todo* el archivo. Sin él, un programa con símbolos sobrantes después de la última clase se aceptaría, porque `<ListaClases>` puede derivar a vacío y el análisis terminaría dejando tokens sin leer. En el léxico de la Etapa 1, **eof** es un token con lexema `$`.

`<ListaClases>` usa **recursión a derecha**, que es la forma natural de expresar «cero o más» en una gramática destinada al análisis descendente. La variante con recursión a izquierda generaría el mismo lenguaje y sería inservible.

`<Clase>` ::= **class idClase** `<GenericidadOpcional>` `<HerenciaOpcional>` **{**
`<ListaMiembros>` **}**

`<Interfaz>` ::= **interface idClase** `<GenericidadOpcional>` `<ExtensionOpcional>` **{**
`<ListaMetodosInterfaz>` **}**

`<GenericidadOpcional>` ::= **< idGen >** | ϵ

`<HerenciaOpcional>` ::= **extends** `<TipoReferencia>` | **implements**
`<TipoReferencia>` | ϵ

`<ExtensionOpcional>` ::= **extends** `<TipoReferencia>` | ϵ

Clase e interfaz son casi simétricas, con tres diferencias deliberadas. Primero, la interfaz sólo admite **extends** y la clase admite **extends** o **implements**: por eso hay dos no terminales distintos en vez de uno compartido, y la restricción queda impuesta *por la gramática* en lugar de delegarse al análisis semántico. Segundo, la interfaz sólo contiene métodos. Tercero, `<HerenciaOpcional>` tiene tres alternativas y no una combinación, lo que impide `extends A implements B` — válido en Java, no en MiniJava.

`<GenericidadOpcional>` admite un **único idGen**, no una lista: MiniJava limita a un solo parámetro de tipo, y el logro *Genericidad Avanzada E2* confirma que esa limitación se mantiene.

3.2 Miembros

`<Miembro>` ::= `<Atributo>` | `<Metodo>` | `<Constructor>`

<Atributo> ::= <Tipo> idMetVar ;

<Metodo> ::= <ModificadorOpcional> <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<MetodoInterfaz> ::= <TipoMetodo> idMetVar <ArgsFormales> ;

<Constructor> ::= public idClase <ArgsFormales> <Bloque>

<ModificadorOpcional> ::= static | ϵ

<TipoMetodo> ::= <Tipo> | void

El public del constructor **no es un modificador de visibilidad**: es lo que sintácticamente lo distingue de todo lo demás. Por eso ni el atributo ni el método lo llevan. Este detalle importa: el logro *Visibilidad Mejorada E2* le saca esa función al public y con eso crea un conflicto nuevo (sección 6.2).

<Tipo> y <TipoMetodo> están separados sólo para que void sea un tipo de retorno válido y no un tipo de variable válido: así `void x;` es imposible sin ningún chequeo adicional.

<MetodoInterfaz> termina en ; y no tiene cuerpo. La diferencia con <Metodo> está en el último símbolo, y como se usan desde no terminales distintos nunca compiten.

3.3 Tipos

<Tipo> ::= <TipoBase> <DimensionesOpcionales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | idGen

<DimensionesOpcionales> ::= [ ] <DimensionesOpcionales> | ϵ

<TipoReferencia> ::= idClase <TipoGenericoOpcional>

<TipoPrimitivo> ::= boolean | char | int

<TipoGenericoOpcional> ::= < <InstanciadoOParametrico> > | ϵ

<InstanciadoOParametrico> ::= idGen | idClase

Las tres familias de <TipoBase> se distinguen con un solo token porque **el léxico ya separó** idClase de idGen — un idGen es una única letra mayúscula. Es un buen ejemplo de una decisión léxica que le simplifica la vida al analizador sintáctico.

<InstanciadoOParametrico> nombra los dos usos de los ángulos: *instanciado* (`Lista<Persona>`, se fija un tipo concreto) y *paramétrico* (`Lista<T>` dentro de una clase genérica, se propaga el parámetro). No admite anidamiento ni notación diamante: las dos cosas las agrega un logro.

Decisión — transcripción de `[]`

El documento de la cátedra escribe [] como si fuera un símbolo, pero léxicamente son **dos** tokens: *Aspectos Léxicos de MiniJava* no define ningún token `[]`. En este informe se escribe [ ], que es lo que el analizador tiene que reconocer. Es la única desviación deliberada de la literalidad del documento original.

3.4 Sentencias

<Bloque> ::= { <ListaSentencias> }

<Sentencia> ::= ; | <Asignacion> ; | <Llamada> ; | <VarLocal> ; | <Return> ;

<Sentencia> ::= <If> | <While> | <Bloque>

```
<Asignacion> ::= <Expresion>
<Llamada> ::= <Expresion>
<VarLocal> ::= var idMetVar = <ExpresionCompuesta>
<Return> ::= return <ExpresionOpcional>
<If> ::= if ( <Expresion> ) <Sentencia>
<If> ::= if ( <Expresion> ) <Sentencia> else <Sentencia>
<While> ::= while ( <Expresion> ) <Sentencia>
```

Las cuatro primeras alternativas de `<Sentencia>` llevan el `;` explícito para que quede visible en la regla; las tres últimas no lo llevan porque terminan en `}` o en otra sentencia. La alternativa `;` sola es la sentencia vacía.

`<VarLocal>` declara **sólo con `var`** y **siempre** con inicialización: en la MiniJava base no existe `int x;`. Además usa `<ExpresionCompuesta>` y no `<Expresion>` a la derecha del `=`, lo que impide `var x = y = 3;`. Es una regla más restrictiva a propósito.

3.5 Expresiones

Conviene leerlas como una pila de capas, cada una llamando a la de abajo. Ese encadenamiento es lo que fija qué se agrupa con qué.

```
<Expresion> ::= <ExpresionCompuesta> <OperadorAsignacion>
    <ExpresionCompuesta>
<Expresion> ::= <ExpresionCompuesta>
<OperadorAsignacion> ::= =
<ExpresionCompuesta> ::= <ExpresionCompuesta> <OperadorBinario>
    <ExpresionBasica>
<ExpresionCompuesta> ::= <ExpresionBasica>
<OperadorBinario> ::= || | && | == | != | < | > | <= | >= | + | - |
    * | / | %
<ExpresionBasica> ::= <OperadorUnario> <Operando> | <Operando>
<OperadorUnario> ::= + | - | !
<Operando> ::= <Primitivo> | <Referencia>
<Primitivo> ::= true | false | intLiteral | charLiteral | null
```

Dos observaciones que condicionan todo lo que sigue.

La asignación no es asociativa. El lado derecho del `=` es una `<ExpresionCompuesta>`, no una `<Expresion>`. En consecuencia `a = b = c` **no es un programa MiniJava válido**. Es fácil pasarlo por alto y «mejorarlo» al transformar; hacerlo agrandaría el lenguaje.

No hay niveles de precedencia entre operadores binarios. Los trece operadores están en una única alternancia plana, sin la jerarquía habitual de un no terminal por nivel. Por lo tanto `a + b * c` y `a * b + c` se derivan con la misma forma: la precedencia y la asociatividad no están en la gramática de MiniJava, y darles lugar sería cambiar el lenguaje. Como corolario, eliminar la recursión a izquierda de `<ExpresionCompuesta>` no pierde nada, porque no había precedencia que perder.

3.6 Referencias y primarios

```
<Referencia> ::= <Primario> | <Referencia> <VarEncadenada> | <Referencia>
    <MetodoEncadenado> | <Referencia> <AccesoArreglo>
<Primario> ::= this | stringLiteral | <AccesoVar> | <CrearArreglo>
<Primario> ::= <LlamadaConstructor> | <LlamadaMetodo> |
    <LlamadaMetodoEstatico> | <ExpresionParentizada>
<AccesoVar> ::= idMetVar
<LlamadaMetodo> ::= idMetVar <ArgsActuales>
<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>
<CrearArreglo> ::= new <TipoBase> <DimensionesConTamanio>
<LlamadaConstructor> ::= new <TipoReferencia> <ArgsActuales>
<ExpresionParentizada> ::= ( <Expresion> )
<VarEncadenada> ::= . idMetVar
<MetodoEncadenado> ::= . idMetVar <ArgsActuales>
<AccesoArreglo> ::= [ <Expresion> ]
```

`<Referencia>` modela una cadena como `this.lista.get(3).nombre[0]`: se arranca de un `<Primario>` y se le cuelgan eslabones. La recursión a izquierda expresa que la cadena se asocia hacia la izquierda, que es lo correcto: `a.b.c` es `(a.b).c`.

`stringLiteral` está en `<Primario>` y no en `<Primitivo>` **a propósito**: de un *String* se pueden colgar eslabones (`"hola".length()`) y de un `intLiteral` no. Por el mismo motivo está `<ExpresionParentizada>`, que habilita `(a + b).metodo()`.

Ningún `<Primario>` deriva un `idClase` suelto: no existe la referencia a una clase por sí sola, siempre tiene que ser `C.metodo(...)`. Ese hecho se usa más adelante.

3.7 Construcciones que faltan

Tres ausencias, que no son errores sino el espacio que dejan los logros:

- `++` y `--` son tokens del léxico — *Aspectos Léxicos* §2.4.2 los lista — pero **no aparecen en ninguna regla**. `<OperadorUnario>` es sólo `+` | `-` | `!`. Son tokens sin lugar hasta el logro *Operadores Posifijos E2*.
- `float` y `floatLiteral` tampoco tienen lugar, y el léxico de la Etapa 1 los emite por un logro de aquella etapa. Sin extender la gramática, ese logro queda inutilizable.
- `for`, `private`, `?` y `->` no están **ni en la gramática ni en el léxico**: los agregan cuatro de los logros de esta etapa, y por lo tanto exigen tocar también el analizador léxico.

4 Diagnóstico

La gramática de la cátedra **no es LL(1)**. Verificando la condición $\text{Primeros}(\beta_i) \cap \text{Primeros}(\beta_j) = \varnothing$ sobre todas las producciones aparecen **dieciséis conflictos** repartidos en **diez no terminales**.

#	No terminal	Defecto	Intersección
1	`<Miembro>`	prefijo común profundo	**boolean**, **char**, **int**, **idClase**, **idGen**
2	`<ListaArgsFormales>`	recursión a izquierda	ídem
3	`<Sentencia>`	**ambigüedad**	todos los de `<Expresion>`
4	`<If>`	prefijo común	**if**
5	`<Expresion>`	prefijo común	todos los de `<ExpresionCompuesta>`
6	`<ExpresionCompuesta>`	recursión a izquierda	todos los de `<ExpresionBasica>`
7–12	`<Referencia>`	recursión a izquierda, tres alternativas (seis pares)	todos los de `<Primario>`
13	`<Primario>`	prefijo común	**idMetVar**
14	`<Primario>`	prefijo común a dos niveles	**new**
15	`<DimensionesConTamanio>`	prefijo común	**[**
16	`<ListaExps>`	prefijo común	todos los de `<Expresion>`

4.1 Cuántos de esos dieciséis son ambigüedades

Siguiendo el criterio de la sección 2.2, se clasificó cada par. El resultado es que **sólo dos** de los dieciséis son ambigüedades reales; los otros catorce son conflictos de predicción que se corrigen mecánicamente sin alterar el lenguaje.

Par	Longitudes derivables	Veredicto
`<VarEncadenada>` / `<MetodoEncadenado>`	2..2 vs. 4..∞	no ambiguo
`<AccesoVar>` / `<LlamadaMetodo>`	1..1 vs. 3..∞	no ambiguo
`<CrearArreglo>` / `<LlamadaConstructor>`	se solapan	no ambiguo (argumento posicional)
`<Atributo>` / `<Metodo>`	se solapan	no ambiguo (argumento posicional)
`<Asignacion>` / `<Llamada>`	se solapan	**ambiguo**
las dos producciones de `<If>`	se solapan	**ambiguo**

El caso de `<VarEncadenada>` y `<MetodoEncadenado>` es el más ilustrativo y el que más se confunde. `<VarEncadenada>` deriva **exactamente** la cadena **. idMetVar**: dos tokens, sin alternativas. `<MetodoEncadenado>` es **. idMetVar** `<ArgsActuales>`, y `<ArgsActuales>` **no es vaciable** — siempre aporta al menos **(** y **)** — de modo que deriva cuatro tokens como mínimo. Los rangos no se solapan, **ninguna cadena puede estar**

en los dos lenguajes y por lo tanto ninguna cadena tiene dos árboles. Comparten prefijo izquierdo; eso rompe LL(1) y se arregla factorizando, pero no es ambigüedad.

Para los dos pares cuyos rangos sí se solapan el argumento es posicional en lugar de por longitud: `<CrearArreglo>` y `<LlamadaConstructor>` comparten **new** y el tipo, pero `<DimensionesConTamanio>` siempre arranca con **[** y `<ArgsActuales>` siempre con **(**, así que las cadenas difieren en el token inmediatamente posterior al tipo; `<Atributo>` y `<Metodo>` comparten `<Tipo>` **idMetVar** y siguen con **;** y con **(** respectivamente.

4.2 Las dos ambigüedades

Ambigüedad — `<Asignacion>` y `<Llamada>` son la misma producción

```
<Asignacion> ::= <Expresion>
<Llamada> ::= <Expresion>
```

Los cuerpos son **idénticos**, y `<Sentencia>` los usa en dos alternativas también idénticas. En consecuencia *toda* sentencia-expresión tiene dos árboles de derivación — uno por `<Asignacion>` y otro por `<Llamada>` — y la gramática no ofrece ningún criterio para elegir. Presumiblemente están así para documentar la intención: en una sentencia-expresión uno espera una asignación o una llamada. Pero sintácticamente `x = 5` y `f()` son las dos una `<Expresion>`, y la gramática no puede distinguirlas.

Ambigüedad — el *dangling else*

En `if (a) if (b) s1; else s2;` el **else** puede ligar al **if** interno o al externo: dos árboles para la misma cadena. Es la ambigüedad clásica, presente tal cual en la gramática.

Ambas se resuelven en la sección 5, y en los dos casos hay que *decidir* un significado, no sólo reescribir.

4.3 Una tercera ambigüedad heredada, y una que agrega un logro

El criterio LL(1) de la cátedra —disjunción de Primeros— es el que decide si el analizador puede predecir, y es el único que interviene en la implementación. Pero el checklist de aceptación de Clase 4, slide 41, tiene un ítem más: que la gramática **no sea ambigua, o sea arreglable**. Verificarlo requiere mirar, además, qué puede seguir a un no terminal anulable. Se hizo esa verificación sobre las 1024 configuraciones, y apareció una ambigüedad más además de las dos anteriores.

Decisión — los Siguientes son un diagnóstico, no parte del método

Detectar una ambigüedad exige mirar el contexto en que aparece un no terminal anulable. Ese cálculo se hizo **una sola vez y fuera del compilador**, con la herramienta descripta en la sección 11, con el único fin de auditar el ítem del checklist.

El analizador decide sólo con Primeros, como enseña la cátedra: los Siguientes no aparecen en la gramática, ni en la elección de producciones, ni en una línea del código. Una vez detectada, cada ambigüedad se resuelve con una regla de implementación —la vía que autoriza Clase 4, slide 49— y no vuelven a intervenir.

Tercera ambigüedad: `new int[3][4]`, heredada de la cátedra. Después de crear un arreglo, un **[** puede ser otra dimensión de la creación o un acceso al arreglo

recién creado. Las dos derivaciones existen ya en la gramática original, con producciones que están todas en el documento:

Creación 2-D: `<CrearArreglo>` ⇒ **new** `<TipoBase>` `<DimensionesConTamanio>` ⇒ **new int [**3**] [**4**]**

Creación 1-D indexada: `<Referencia>` ⇒ `<Referencia>` `<AccesoArreglo>` ⇒ `<Primario>` **[**4**]** ⇒ **new int [**3**] [**4**]**

La cuarta: el cuerpo de la lambda. `x -> a = b` puede leerse `x -> (a = b)` —el cuerpo se come la asignación— o `(x -> a) = b`. Afecta a un solo no terminal, `<RestoExpresion>`, que es la única cola externa que todavía compite con la cola del cuerpo. Se resuelve voraz, que es lo que hace Java y lo que un descendente recursivo hace por construcción: el método del cuerpo llama a `<Expresion>` y ésta avanza mientras pueda.

Decisión — dónde vive la lambda: no es un operando

La ubicación que se impone sola al leer la especificación del logro es `<Primario>`: la lambda aparece donde aparece un valor, y `<Primario>` es el no terminal de los valores. Se evaluó y se descartó, porque `<Primario>` es la ranura de un *operando* y eso trae dos consecuencias, la segunda grave:

1. la cola de la expresión que la contiene competía con la cola de su cuerpo, y la ambigüedad anterior se multiplicaba por cinco: `<RestoExpresion>` (=), `<RestoExpresionCompuesta>` (los trece binarios), `<RestoReferencia>` (. y **[**), `<RestoCondicional>` (**?**) y `<PosfijoOpcional>` (++, --);
2. **el lenguaje reconocido era estrictamente más grande que el de Java**. Si una lambda es un `<Primario>`, entonces es un operando válido, y `1 + x -> x`, `a == x -> x`, `- x -> x`, `(x -> x) + 1` y `(x -> x).m()` pasan el análisis. En Java los cinco son error —*«lambda expression not expected here»*, verificado con `javac`—, porque el JLS define *Expression: LambdaExpression | AssignmentExpression*: la lambda es una alternativa del nivel **más alto** de la jerarquía, nunca un *Primary*. Dicho de otro modo: -> no tiene precedencia contra los operadores porque no compite con ellos.

Por la regla «lo que no está especificado se comporta como en Java», el segundo punto decide: la lambda se produce en el nivel de `<Expresion>`, como en el JLS. El costo es factorizar las dos cabezas que comparte con una expresión normal —**idMetVar** y **(**— contra la cola de expresión, con la misma técnica que `<ExpresionSinIdClase>`: catorce producciones nuevas, ninguna lógica duplicada, porque todas las colas se comparten. El beneficio es doble: el lenguaje pasa a ser el de Java en las cinco formas de la tabla, y cuatro de las cinco ambigüedades **desaparecen de raíz** en vez de resolverse por regla.

5 Transformaciones

5.1 Orden de trabajo y criterios

Primero eliminar la recursión a izquierda, después factorizar. El orden importa porque eliminar la recursión genera producciones nuevas sin factorizar; la sección 5.2.3 es el caso concreto donde eso ocurre.

No factorizar de más. Las reglas originales tienen significado y el analizador semántico de la Etapa 3 lo va a necesitar. En la práctica eso se tradujo en cuatro criterios:

- se conserva el nombre original siempre que la regla sobreviva;
- los no terminales nuevos llevan **nombres semánticos en español** (`<RestoExpresionCompuesta>`, no `<ExpresionCompuesta'>`);
- cuando dos no terminales originales se fusionan a la fuerza, se dice cuáles y por qué;
- ante la disyuntiva entre fusionar y **insertar un no terminal**, se inserta.

Decisión — insertar el no terminal, no fusionar

Al factorizar un prefijo hay dos caminos. *Fusionar* es meter los cuerpos de los no terminales involucrados dentro del padre y borrarlos: quedan menos no terminales, pero el punto donde se decide *qué construcción era* pierde el nombre, y el analizador semántico pierde el lugar donde colgar el nodo del árbol. *Insertar un no terminal* es factorizar el prefijo y poner la disyuntiva restante en un no terminal nuevo y dedicado, cuyo nombre dice qué se está decidiendo: un no terminal más, pero se traduce a un método del analizador que **es** el punto de decisión semántica.

Se adopta siempre la segunda, con dos corolarios. **Un no terminal de decisión por contexto, aunque la forma coincida**: dos disyuntivas con la misma pinta pero distinto significado no se comparten. Y **compartir sí cuando es literalmente la misma construcción**: `<CuerpoDeMetodo>` se usa desde método, constructor y método **static** porque en los tres casos es lo mismo.

5.2 Eliminación de recursión a izquierda

El esquema general:

$$A \to A\alpha_1 \mid \cdots \mid A\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m \quad \Longrightarrow \quad \begin{cases} A \to \beta_1 R \mid \cdots \mid \beta_m R \\ R \to \alpha_1 R \mid \cdots \mid \alpha_n R \mid \epsilon \end{cases}$$

5.2.1 R1 — `<ListaArgsFormales>`

1. Original

```
<ListaArgsFormales> ::= <ArgFormal>
<ListaArgsFormales> ::= <ListaArgsFormales> , <ArgFormal>
```

2. Defecto Recursión a izquierda directa. Ambas alternativas tienen Primeros(`<ArgFormal>`) = Primeros(`<Tipo>`) = {**boolean**, **char**, **int**, **idClase**, **idGen**}. Un analizador recursivo que intentara la segunda producción se llamaría a sí mismo sin haber consumido ningún token: recursión infinita.

3. Transformación Esquema directo, con α_1 = `, <ArgFormal>` y β_1 = `<ArgFormal>`.

4. Resultado

```
<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>
<RestoListaArgsFormales> ::= , <ArgFormal> <RestoListaArgsFormales> | ϵ
```

{,} contra {ϵ}: disjuntos.

5.2.2 R2 — `<ExpresionCompuesta>`

1. Original

```
<ExpresionCompuesta> ::= <ExpresionCompuesta> <OperadorBinario>
    <ExpresionBasica>
<ExpresionCompuesta> ::= <ExpresionBasica>
```

2. Defecto Recursión a izquierda directa; es el caso canónico $E \to E + T \mid T$.

3. Transformación Esquema directo.

4. Resultado

```
<ExpresionCompuesta> ::= <ExpresionBasica> <RestoExpresionCompuesta>
<RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>
    <RestoExpresionCompuesta> | ϵ
```

Los trece operadores binarios contra $\{\epsilon\}$: disjuntos. Como se señaló, la gramática original ya era plana en precedencia, así que la transformación no pierde nada.

5.2.3 R3 — `<Referencia>`, y el conflicto que la transformación genera

1. Original

```
<Referencia> ::= <Primario>
<Referencia> ::= <Referencia> <VarEncadenada>
<Referencia> ::= <Referencia> <MetodoEncadenado>
<Referencia> ::= <Referencia> <AccesoArreglo>
```

2. Defecto Recursión a izquierda con *tres* alternativas recursivas: las cuatro producciones tienen los Primeros de `<Primario>`, lo que da los seis pares en conflicto.

3. Transformación Esquema directo con β_1 = `<Primario>` y los tres eslabones como α_i.

4. Resultado intermedio, con un defecto nuevo

```
<Referencia> ::= <Primario> <RestoReferencia>
<RestoReferencia> ::= <VarEncadenada> <RestoReferencia> |
    <MetodoEncadenado> <RestoReferencia> | <AccesoArreglo>
    <RestoReferencia> | ϵ
```

Primeros(`<VarEncadenada>`) = Primeros(`<MetodoEncadenado>`) = {`.`}: **siguen chocando**. Es exactamente el fenómeno que obliga a hacer la recursión antes que la factorización.

5. Factorización del defecto nuevo Los dos comparten *todo* el prefijo y sólo se distinguen por si después viene un `(`. Se fusionan, y la disyuntiva va a un no terminal dedicado.

```
<Referencia> ::= <Primario> <RestoReferencia>
<RestoReferencia> ::= <Encadenado> <RestoReferencia> | <AccesoArreglo>
    <RestoReferencia> | ϵ
<Encadenado> ::= . idMetVar <EncadenadoAtributoOMetodo>
<EncadenadoAtributoOMetodo> ::= <ArgsActuales> | ϵ
```

{`.`}, {`[`}, $\{\epsilon\}$: disjuntos.

Decisión — costo semántico de R3, declarado

Se pierde la distinción entre `<VarEncadenada>` (acceso a un atributo de otro objeto) y `<MetodoEncadenado>` (llamada encadenada). Es una pérdida **inevitable**: sintácticamente son indistinguibles hasta ver el token siguiente. Queda mitigada por el no terminal insertado: `<EncadenadoAtributoOMetodo>` *es* el punto donde el analizador decide cuál de las dos era, según haya derivado a vacío o no. El nombre lo dice.

5.3 Factorización a izquierda

$$X \to \alpha\beta_1 \mid \cdots \mid \alpha\beta_n \mid \gamma_1 \mid \cdots \mid \gamma_m \quad \Longrightarrow \quad \begin{cases} X \to \alpha R \mid \gamma_1 \mid \cdots \mid \gamma_m \\ R \to \beta_1 \mid \cdots \mid \beta_n \end{cases}$$

5.3.1 F1 — `<Expresion>`

1–2 Prefijo común α = `<ExpresionCompuesta>`: viendo `x`, el analizador no sabe si va a aparecer un `=`, y puede estar arbitrariamente lejos.

3–4. Resultado

```
<Expresion> ::= <ExpresionCompuesta> <RestoExpresion>
<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCompuesta> | ϵ
```

Decisión — la transformación «natural» de F1 agranda el lenguaje

La versión que sale sola al escribirla, y que además queda más prolija, es `<RestoExpresion> ::= <OperadorAsignacion> <Expresion> | ϵ`. **Es incorrecta**: la regla original tiene `<ExpresionCompuesta>` a la derecha del `=`, o sea que la asignación no es asociativa y `a = b = c` no pertenece al lenguaje. Poner `<Expresion>` la volvería asociativa a derecha y aceptaría programas que la cátedra no define.

Una transformación sólo es válida si **preserva exactamente** el lenguaje generado. El mismo cuidado se aplicó en F4.

5.3.2 F2 — `<Sentencia>`: resolver la ambigüedad

1–2 No es un prefijo común sino la ambigüedad descripta en la sección 4: `<Asignacion>` y `<Llamada>` tienen cuerpos idénticos.

3. Transformación No hay factorización que lo arregle: hay que *elegir*. Se colapsan en una única alternativa, con un nombre que dice lo que efectivamente es.

4. Resultado

```
<Sentencia> ::= ; | <SentenciaExpresion> ; | <VarLocal> ; | <Return> ; | <If>
    | <While> | <Bloque>
<SentenciaExpresion> ::= <Expresion>
```

Decisión — distinguir asignación de llamada es trabajo del semántico

Colapsar las dos alternativas **no achica el lenguaje**: lo genera exactamente igual, con un árbol por sentencia en lugar de dos. Determinar si la expresión es una asignación válida, una llamada válida o ninguna de las dos — `5 = x;` atraviesa el análisis sintáctico — requiere conocer el significado de la expresión, y eso es análisis semántico.

5.3.3 F3 — `<If>`: factorizar, y después resolver el *dangling else*

1–2 Prefijo común larguísimo: **if (`<Expresion>`) `<Sentencia>`**. El analizador debería atravesar una condición y una sentencia entera antes de saber qué producción eligió.

3–4. Resultado

`<If>` ::= **if (** `<Expresion>` **)** `<Sentencia>` `<ElseOpcional>`

`<ElseOpcional>` ::= **else** `<Sentencia>` | ϵ

{**else**} contra {ϵ}: disjuntos.

5. Lo que la factorización no arregla La ambigüedad del *dangling else* sigue ahí, escondida en que `<ElseOpcional>` puede derivar a ϵ *y* tener un **else** en lo que le sigue.

Decisión — el *dangling else* se resuelve con una regla de implementación

Es la segunda de las dos soluciones posibles ante una ambigüedad: en lugar de reescribir la gramática, se agrega una regla en la implementación del analizador. El esquema de traducción toma la rama **else** apenas ve el token, y como el **if** que se está ejecutando en ese momento es el más interno, el **else** liga al **if** más cercano — **que es exactamente la regla de Java**. No hace falta tocar la gramática: alcanza con dejar constancia de que el orden de las ramas en el método de `<ElseOpcional>` es significativo.

5.3.4 F4 — `<DimensionesConTamanio>`

1–2 Prefijo común **[** `<Expresion>` **]**.

3. Transformación **Con cuidado**: la regla original exige *al menos una* dimensión. La versión `<DimensionesConTamanio>` ::= **[** `<Expresion>` **]** `<DimensionesConTamanio>` | ϵ permitiría cero y agrandaría el lenguaje. La parte repetible va a un no terminal opcional aparte.

4. Resultado

`<DimensionesConTamanio>` ::= **[** `<Expresion>` **]** `<DimensionesConTamanioOpcional>`

`<DimensionesConTamanioOpcional>` ::= `<DimensionesConTamanio>` | ϵ

5.3.5 F5 — `<ListaExps>`

Prefijo común `<Expresion>`. Se factoriza con la misma forma que R1, para que las dos listas de la gramática se lean igual.

`<ListaExps>` ::= `<Expresion>` `<RestoListaExps>`

`<RestoListaExps>` ::= **,** `<Expresion>` `<RestoListaExps>` | ϵ

5.3.6 F6 — `<Miembro>`: factorización profunda

1. Original Ver sección 5; las tres alternativas son `<Atributo>`, `<Metodo>` y `<Constructor>`.

2. Defecto Prefijo común **no directo**: mirando sólo las tres alternativas de `<Miembro>` no se ve nada compartido, pero al bajar a sus producciones aparece que `<Atributo>` y

<Metodo> empiezan los dos con <Tipo>.

$$\text{Primeros}(\texttt{<Atributo>}) = \text{Primeros}(\texttt{<Tipo>}) = \{\textbf{boolean}, \textbf{char}, \textbf{int}, \textbf{idClase}, \textbf{idGen}\}$$
$$\text{Primeros}(\texttt{<Metodo>}) = \{\textbf{static}, \textbf{void}\} \cup \text{Primeros}(\texttt{<Tipo>})$$

Ante `int x` el analizador no sabe si sigue un **;** o un **(**, y eso está dos tokens más adelante.

3. Transformación Factorización **profunda**: subir las producciones de <Atributo> y <Metodo> hasta <Miembro>, expandir <ModificadorOpcional> y <TipoMetodo> para exponer el prefijo, y recién ahí hacer factor común. Expandiendo:

<Miembro> ::= <Tipo> **idMetVar ;**

<Miembro> ::= **static** <TipoMetodo> **idMetVar** <ArgsFormales> <Bloque>

<Miembro> ::= <Tipo> **idMetVar** <ArgsFormales> <Bloque>

<Miembro> ::= **void idMetVar** <ArgsFormales> <Bloque>

<Miembro> ::= **public idClase** <ArgsFormales> <Bloque>

Ahora el prefijo <Tipo> **idMetVar** sí es directo, entre la primera y la tercera.

4. Resultado

<Miembro> ::= <Tipo> **idMetVar** <AtributoOMetodo>

<Miembro> ::= **static** <TipoMetodo> <Metodo>

<Miembro> ::= **void** <Metodo>

<Miembro> ::= <Constructor>

<AtributoOMetodo> ::= **;** | <CuerpoDeMetodo>

<Metodo> ::= **idMetVar** <CuerpoDeMetodo>

<CuerpoDeMetodo> ::= <ArgsFormales> <Bloque>

<Constructor> ::= **public idClase** <CuerpoDeMetodo>

{**boolean**, **char**, **int**, **idClase**, **idGen**}, {**static**}, {**void**}, {**public**}: disjuntos. Y en <AtributoOMetodo>, {**;**} contra {**(**}: disjuntos.

<Constructor> **sobrevive como no terminal propio** porque **public** lo distingue solo. Se extraen además dos no terminales compartidos: <CuerpoDeMetodo> (argumentos formales más bloque), que es literalmente la misma construcción en método, método **static** y constructor; y <Metodo>, que comparten la rama **static** y la rama **void**, de modo que el analizador semántico tiene un único lugar donde construir el nodo de método para esas dos formas. La tercera forma — la de tipo de referencia — ya consumió el **idMetVar** al factorizar contra el atributo, y por eso llega por <AtributoOMetodo>.

5.3.7 F7 — <Primario>: el prefijo idMetVar

<AccesoVar> (**idMetVar**) y <LlamadaMetodo> (**idMetVar** <ArgsActuales>) comparten todo el prefijo. Misma situación que en R3 y misma resolución, con un no terminal de decisión **dedicado a este contexto**:

<AccesoVarOLlamada> ::= **idMetVar** <AccesoLlamada>

<AccesoLlamada> ::= <ArgsActuales> | ϵ

Decisión — un punto de decisión por contexto, aunque la forma coincida

Una versión anterior de esta transformación resolvía R3 y F7 con el *mismo* no terminal <ArgsActualesOpcional>, y se había justificado como una buena abstracción. Es al revés: su ϵ significa dos cosas distintas según quién lo llame — desde

`<Encadenado>` era un `<VarEncadenada>`, acceso a un atributo de *otro* objeto; desde `<AccesoVarOLlamada>` era un `<AccesoVar>`, acceso a una variable local, un parámetro o un atributo propio. Son dos nodos distintos, con reglas de resolución de nombres distintas, y un único método del analizador tendría que preguntarse quién lo llamó para saber cuál construir. Se separaron en `<EncadenadoAtributoOMetodo>` y `<AccesoLlamada>`: cuesta un no terminal más y deja los cuatro no terminales originales recuperables desde el nombre del método que decide.

5.3.8 F8 — `<Primario>`: el prefijo new, a dos niveles

2. Defecto `<CrearArreglo>` y `<LlamadaConstructor>` comparten **new**, y no alcanza con factorizarlo porque `<TipoBase>` puede derivar a `<TipoReferencia>`: `new C` sigue sin decidirse. El token que desempata es el que viene *después* del tipo.

3–4. Resultado Se factoriza **new**, se expande `<TipoBase>` para separar los casos que sí se deciden con un token, y se factoriza una segunda vez el caso **idClase**.

```
<Creacion> ::= new <ArregloOInstancia>
<ArregloOInstancia> ::= <TipoPrimitivo> <DimensionesConTamanio>
<ArregloOInstancia> ::= idGen <DimensionesConTamanio>
<ArregloOInstancia> ::= idClase <TipoGenericoOpcional>
    <ArregloOInstanciaDeClase>
<ArregloOInstanciaDeClase> ::= <DimensionesConTamanio> | <ArgsActuales>
```

`new int[3]` y `new T[3]` se resuelven en el primer token después de **new**; sólo `new C...` necesita el segundo nivel. La factorización se hizo **sólo donde hacía falta**.

5.4 Estado del núcleo

Con R1–R3 y F1–F8, el núcleo de la gramática — sin logros — es LL(1). El costo respecto de la base fue de **seis no terminales y ocho producciones**, concentrado en los `<Resto...>` y en los puntos de decisión insertados, que el esquema de traducción convierte en un `if/else` cada uno.

6 Extensiones por logro

Se persiguen **los nueve logros de la Etapa 2 que afectan al analizador**, más el *carry-over* de los floats de la Etapa 1. Cada uno se transforma con el mismo procedimiento de cuatro pasos; la diferencia con el núcleo es que acá los conflictos **no vienen de la gramática de la cátedra: los crea el logro**.

6.1 La condición de atomicidad

El compilador debe poder ejecutarse y probarse con cualquier subconjunto de logros, incluido el vacío. A nivel de gramática eso se traduce en una condición dura:

> **Para todo subconjunto de logros, la gramática resultante debe ser LL(1).**

No alcanza con que lo sea con todos prendidos. La condición tiene una consecuencia de diseño concreta: **ningún logro puede apoyarse en un no terminal que introdujo otro**. Cuando dos logros necesitan una construcción parecida, cada uno lleva la suya — ver sección 6.3.

Como técnica, conviene que **desactivar un logro sea quitar alternativas y no cambiar de estructura**. Los logros de ternario y de operadores posfijos son el ejemplo limpio: apagados, sus no terminales quedan en sólo ϵ y la capa desaparece sin tocar ninguna otra regla.

6.2 Visibilidad Mejorada E2

Defecto que crea **public** y **private** pasan a poder encabezar cualquier miembro. El **public** del `<Constructor>` original queda absorbido, el constructor se vuelve **idClase** `<CuerpoDeMetodo>` y **empieza a competir** con el atributo o método de tipo referencia por el token **idClase**: `C(){}` contra `C x;`.

Transformación Factorizar la visibilidad al frente y, adentro, factorizar el **idClase**. El desempate es el token que sigue: **(** es constructor, cualquier otro abre un tipo.

```
<Miembro> ::= <VisibilidadOpcional> <MiembroSinVisibilidad>
<VisibilidadOpcional> ::= public | private | ϵ
<MiembroSinVisibilidad> ::= static <TipoMetodo> idMetVar <CuerpoDeMetodo>
<MiembroSinVisibilidad> ::= void idMetVar <CuerpoDeMetodo>
<MiembroSinVisibilidad> ::= <TipoPrimitivo> <DimensionesOpcionales>
    idMetVar <AtributoOMetodo>
<MiembroSinVisibilidad> ::= idGen <DimensionesOpcionales> idMetVar
    <AtributoOMetodo>
<MiembroSinVisibilidad> ::= idClase <MiembroDesdeIdClase>
<MiembroDesdeIdClase> ::= <CuerpoDeMetodo>
<MiembroDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales>
    idMetVar <AtributoOMetodo>
```

{**static**}, {**void**}, {**boolean**, **char**, **int**, **float**}, {**idGen**}, {**idClase**}: disjuntos. Y en `<MiembroDesdeIdClase>`, {**(**} contra {<, **[**, **idMetVar**}: disjuntos.

Con el logro apagado vuelve la forma del núcleo y `<MiembroDesdeIdClase>` se queda con una sola producción.

Decisión — en los métodos de interfaz se admite public y no private

El enunciado del logro habla de «Métodos, Atributos y Constructores», y `<MetodoInterfaz>` es un no terminal aparte de `<Miembro>`, así que la visibilidad de los métodos de interfaz cae bajo la regla general de la cátedra: lo que no está especificado se comporta como en Java. Verificado contra `javac`:

En Java	Resultado
`interface A { int m(); }`	compila
`interface B { public int m(); }`	compila — redundante, pero legal
`interface C { private int m(); }`	**error**: *missing method body*
`interface D { private int m() {...} }`	compila

O sea que en Java un método de interfaz **private exige cuerpo**. Y en MiniJava no hay forma de escribirlo: `<MetodoInterfaz>` termina en **;** en la especificación original y **ningún logro de esta etapa lo cambia**, así que los métodos de interfaz son siempre abstractos. Un **private** ahí sería, por construcción, siempre la forma que Java rechaza.

De ahí la asimetría, que no es una preferencia sino una consecuencia:

```
<MetodoInterfaz> ::= <PublicOpcional> <TipoMetodo> idMetVar <ArgsFormales>
    ;
<PublicOpcional> ::= public | ϵ
```

{**public**} contra los Primeros de `<TipoMetodo>`: disjuntos.

6.3 Atributos Inicializados y Variables locales Clásicas E2

Los dos agregan un **=** `<ExpresionCompuesta>` opcional, y la tentación es usar un único no terminal compartido. **Está mal** por la condición de atomicidad: son logros distintos, y con *atrinit* apagado y *varclasica* prendido las variables locales tienen que poder inicializarse igual. Van dos no terminales dedicados, `<InicializacionDeAtributo>` e `<InicializacionDeDeclarador>`, con el mismo cuerpo y distinta marca de logro.

Defecto que crea *varclasica* Una declaración local pasa a poder empezar con **idClase** (`C x;`), y una sentencia-expresión también (`C.m();`, por `<LlamadaMetodoEstatico>`). Chocan.

Transformación Factorizar el **idClase** en `<Sentencia>`: **.** significa expresión, cualquier otro token significa declaración.

```
<Sentencia> ::= <ExpresionSinIdClase> ;
<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales>
    <ListaDeclaradores> ;
<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;
<Sentencia> ::= idClase <SentenciaDesdeIdClase>
<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales>
    <ListaDeclaradores> ;
<SentenciaDesdeIdClase> ::= . idMetVar <ArgsActuales>
    <ColaDeExpresionDesdePrimario> ;
```

La rama de expresión tiene que retomar una expresión *ya empezada*. Escribirla a mano duplicaría media jerarquía de expresiones; en su lugar se insertan tres no terminales que capturan las colas compartidas:

```
<ExpresionSinIdClase> ::= <OperadorUnario> <Operando> <ColaTrasOperando>
<ExpresionSinIdClase> ::= <Primitivo> <ColaTrasOperando>
<ExpresionSinIdClase> ::= <PrimarioSinIdClase>
    <ColaDeExpresionDesdePrimario>
<ColaTrasOperando> ::= <RestoExpresionCompuesta> <RestoCondicional>
    <RestoExpresion>
<ColaDeExpresionDesdePrimario> ::= <RestoReferencia> <PosfijoOpcional>
    <ColaTrasOperando>
<Primario> ::= <LlamadaMetodoEstatico> | <PrimarioSinIdClase>
```

<LlamadaMetodoEstatico> es la única forma de <Primario> que empieza con idClase, lo que hace que la partición sea exacta. **Sólo se duplican las cabezas**: todas las colas se comparten con la jerarquía normal, de modo que no hay lógica repetida y un cambio futuro en las expresiones no hay que espejarlo.

Decisión — alternativa descartada: dos tokens de anticipación

El mismo conflicto se resuelve dándole al analizador *dos* tokens de anticipación sólo en este punto — mirar si después del idClase viene un . — con cero no terminales nuevos. Se descartó porque despega la implementación del esquema de traducción visto en clase, que es de un único token, y ese esquema es la garantía de correctitud que se está siguiendo. El costo de la solución LL(1) pura son tres no terminales sin lógica duplicada, que es barato.

6.4 Fors! E2

Un <For> nuevo como alternativa de <Sentencia>. El encabezado tiene el mismo problema de idClase que la sentencia y se resuelve igual, reutilizando <ExpresionSinIdClase> y <ColaDeExpresionDesdePrimario>. El desempate entre el for estándar y el de iteradores es de un solo token y está justo después del declarador: : es de iteradores, = o ; es estándar.

<For> ::= for (<EncabezadoFor>) <Sentencia>

<EncabezadoFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>

<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar>

<EncabezadoFor> ::= <TipoPrimitivo> <DimensionesOpcionales> idMetVar
<EncabezadoForTrasDeclarador>

<EncabezadoFor> ::= idGen <DimensionesOpcionales> idMetVar
<EncabezadoForTrasDeclarador>

<EncabezadoFor> ::= idClase <EncabezadoForDesdeIdClase>

<EncabezadoFor> ::= <ExpresionSinIdClase> <ClausulasFinalesFor>

<EncabezadoForTrasDeclarador> ::= : <Expresion>

<EncabezadoForTrasDeclarador> ::= <InicializacionDeDeclaradorFor>
<ClausulasFinalesFor>

<ClausulasFinalesFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>

<ClausulasFinalesFor> con una sola <ExpresionOpcional> por separador es la lectura literal de «se restringe a una sola sentencia o expresión por separador».

6.5 Operador Ternario E2

Una capa nueva entre la asignación y los operadores binarios, con la precedencia de Java.

<Expresion> ::= <ExpresionCondicional> <RestoExpresion>

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicional> | ϵ

<ExpresionCondicional> ::= <ExpresionCompuesta> <RestoCondicional>

<RestoCondicional> ::= ? <ExpresionCondicional> : <ExpresionCondicional> | ϵ

Asociativo a **derecha** — a ? b : c ? d : e es a ? b : (c ? d : e) — porque la rama del : vuelve a <ExpresionCondicional>, que puede tener su propio

`<RestoCondicional>`. Con el logro apagado, `<RestoCondicional>` queda en sólo ϵ y `<ExpresionCondicional>` se vuelve un alias de `<ExpresionCompuesta>`: la capa desaparece sin dejar rastro.

Decisión — dónde alcanza el ternario

La gramática original usa `<ExpresionCompuesta>` como lado derecho de un `=` en *los seis lugares* donde aparece uno: la asignación y los cinco inicializadores (variable local con `var`, atributo, declarador clásico, y los dos del encabezado del `for`). Lo hace para impedir la asignación encadenada: `a = b = c` no es un programa MiniJava válido.

Al insertar la capa del ternario *por encima* de `<ExpresionCompuesta>`, esos seis lugares quedarían sin acceso al operador, y `var x = c ? a : b;` sería inválido aunque `x = c ? a : b;` no lo fuera — una asimetría sin ninguna justificación. Los seis pasan entonces a `<ExpresionCondicional>`, que es exactamente un escalón más arriba: sigue sin admitir un `=` y por lo tanto sigue impidiendo la asignación encadenada, pero admite el ternario. Y como con el logro apagado `<ExpresionCondicional>` es un alias de `<ExpresionCompuesta>`, el lenguaje base queda **idéntico** al de la cátedra.

Este logro requiere el token `?`, que **no existe en el léxico de la Etapa 1**.

6.6 Operadores Posifijos E2

`<Operando>` ::= `<Primitivo>` | `<Referencia>` `<PosfijoOpcional>`

`<PosfijoOpcional>` ::= `++` | `--` | ϵ

El posfijo va sobre `<Referencia>` y **no** sobre `<Operando>`: `5++` no tiene sentido. Misma técnica de desactivación que el ternario. Es el único logro que **no** requiere tocar el léxico: `++` y `--` ya son tokens desde la Etapa 1 y no tenían dónde usarse.

6.7 MiniLambdas E2

Defecto que crea Dos, y el segundo es el que da trabajo.

El primero es **dónde vive la lambda**. No es una capa más de la jerarquía de precedencias: el JLS la pone como alternativa de *Expression*, el nivel más alto, nunca como *Primary*. Ponerla en `<Primario>` —la ranura del operando— haría válidos `1 + x -> x`, `- x -> x` y `(x -> x).m()`, que `javac` rechaza con *«lambda expression not expected here»*. El análisis completo está en la sección 4.3.

El segundo es la **anticipación**. El `(` abre tanto una expresión parentizada como la lista de parámetros de una lambda, y `idMetVar` abre tanto un acceso a variable como una lambda de un parámetro. A primera vista `(a, b, c) -> x` contra `(a + b)` exige escanear hasta el `)` para decidir, o sea anticipación ilimitada.

Análisis **No la exige.** Los casos se separan solos:

Entrada	Decisión	Tokens
`( )`	sólo puede ser lambda de cero parámetros	1 (el `)`)
`(` *e* `,` ...	sólo puede ser lambda: no hay operador coma	1 (la `,`)
`(` *e* `)`	lo decide el token *posterior* al `)`	1 (el `->`)
`idMetVar ->`	lo decide el token posterior al identificador	1 (el `->`)

La clave es que el -> viene *inmediatamente* después del), así que en el momento de decidir sigue siendo un token de anticipación.

Resultado Un nivel nuevo por encima de <ExpresionCondicional> que admite lambda y no admite =, que es exactamente el *LambdaExpression* / *ConditionalExpression* del JLS, y donde se factorizan las dos cabezas ambiguas:

```
<Expresion> ::= <ExpresionCondicionalOLambda> <RestoExpresion>
<ExpresionSinLambda> ::= <ExpresionCondicional> <RestoExpresion>
<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicionalOLambda> |
    ϵ
<ExpresionCondicionalOLambda> ::= <OperadorUnario> <Operando>
    <ColaTrasOperandoSinAsignacion>
<ExpresionCondicionalOLambda> ::= <Primitivo>
    <ColaTrasOperandoSinAsignacion>
<ExpresionCondicionalOLambda> ::= <ExpresionDesdePrimarioOLambda>
<ExpresionDesdePrimarioOLambda> ::= idMetVar <AccesoLlamadaOLambda> |
    <ParentizadaOLambda>
<ExpresionDesdePrimarioOLambda> ::= this <ColaDesdePrimarioSinAsignacion>
    | ...
<AccesoLlamadaOLambda> ::= -> <CuerpoLambda> | <AccesoLlamada>
    <ColaDesdePrimarioSinAsignacion>
<ParentizadaOLambda> ::= ( <ContenidoDeParentesis>
<ContenidoDeParentesis> ::= ) -> <CuerpoLambda> | <Expresion>
    <RestoDeParentesis>
<RestoDeParentesis> ::= ) <FinDeParentizadaOLambda>
<RestoDeParentesis> ::= , <ListaParametrosLambda> ) -> <CuerpoLambda>
<FinDeParentizadaOLambda> ::= -> <CuerpoLambda> |
    <ColaDesdePrimarioSinAsignacion>
<CuerpoLambda> ::= <Expresion>
```

Ningún <Primario> puede ya ser una lambda: <Parentizada> ::= (<ExpresionSinLambda>), y <AccesoVarOLlamada> perdió la alternativa del ->. Los seis lugares donde el JLS sí admite una lambda —argumento, índice de arreglo, las dos ramas del ternario, lado derecho de un = y cuerpo de otra lambda— la siguen admitiendo, porque los seis pasan por <Expresion> o por <ExpresionCondicionalOLambda>. <CuerpoLambda> ::= <Expresion> es la lectura literal de «el cuerpo debe estar formado por una única expresión, no se permiten cuerpos delimitados por llaves»: como no hay alternativa con {, la gramática **rechaza** `x -> { ...}` sin ningún chequeo adicional.

Requiere el token ->, que no existe en el léxico de la Etapa 1.

Decisión — permisividad declarada de las lambdas

En `( a , b ) -> x` el `a` se deriva como <Expresion>, no como parámetro, de modo que `( a + 1 , b ) -> x` **atraviesa el análisis sintáctico**. Rechazarlo exige saber que esa expresión tiene que ser un identificador suelto, lo que es un chequeo semántico. Es la misma clase de permisividad que la gramática original ya tiene con `5 = x;`. Lo mismo vale, como se anticipó en la sección 1.2, para la prohibición de capturar variables.

Hay una segunda, del mismo origen. `<ContenidoDeParentesis> ::= <Expresion> <RestoDeParentesis>`, y esa `<Expresion>` puede ser una lambda —`f((x -> x))` es válido en Java—. Pero `(x -> x).m()` no lo es: parentizarla no la convierte en operando. Distinguirlos en la gramática exigiría clonar toda la cadena de colas en su variante «el contenido del paréntesis fue una lambda», unos siete no terminales nuevos, así que lo cierra el analizador con lo que acaba de leer.

Lo que verifica es si la **raíz** de esa expresión es una lambda, no si se vio una lambda en algún lado: `(a = x -> x).m()` es una asignación y `(a ? b : x -> x).m()` un ternario, y Java acepta la forma de los dos. `javac` igual rechaza el segundo, pero por la regla de *poly expressions* del JLS 15.25, que depende de la clasificación de la expresión y de su tipo destino —un chequeo del analizador semántico, no de la forma—. Queda declarado.

Decisión — this y la captura los rechaza el analizador, no la gramática

Las otras dos cláusulas del logro —«no se permite la captura de variables locales, parámetros, ni el uso de this»— **no son expresables en una gramática libre de contexto**: no hay forma de escribir «this es un `<Primario>` válido, salvo debajo de un `<CuerpoLambda>`», ni de condicionar un identificador a una lista que se leyó unas producciones antes. Escribirlo exigiría duplicar la jerarquía de expresiones en variantes «sin this» y «sólo con estos nombres».

Pero **sí son verificables en el analizador**, y con información puramente local:

- **this explícito**: un indicador booleano que se prende al entrar al cuerpo de una lambda y se apaga al salir. El analizador sabe dónde está porque acaba de entrar.
- **this implícito**: llamar a un método propio sin escribirlo — `x -> metodo(x)`— es un uso de this igual. Y dentro de una lambda de MiniJava una llamada sin punto *sólo* puede ser un método de la clase: no hay funciones sueltas y un parámetro es un valor, no se lo puede llamar. Mismo indicador.
- **captura de variables**: el conjunto de nombres legales dentro del cuerpo es exactamente el de los parámetros de la lambda, que el analizador acaba de leer. Como el cuerpo es *una única expresión*, adentro no se puede declarar nada, así que el conjunto es cerrado. Y las lambdas anidadas no necesitan una pila de alcances: usar el parámetro de la lambda de afuera también está prohibido, de modo que alcanza con el conjunto de la más interna.

Decisión — el chequeo de captura se difiere adentro de un paréntesis

Hay un punto donde el chequeo no puede resolverse en el momento. El contenido de un paréntesis se lee como `<Expresion>`, porque hasta ver el token que sigue al) no se sabe si era una expresión o la lista de parámetros de otra lambda. Esa lectura pasa por el chequeo de captura, y ahí todavía rige el alcance de la lambda de afuera: en `x -> (y) -> y` el `y` se tomaría por una captura, y se rechazaría una lambda que escrita `x -> y -> y` se acepta.

Mientras el paréntesis está abierto, entonces, un nombre que no es parámetro no se rechaza: queda **en duda**. Al cerrar se resuelve con el mismo token que ya decide todo lo demás: si sigue un -> eran declaraciones y las dudas se descartan; si no, era una expresión parentizada, eran capturas de verdad y se reporta la primera en orden de aparición. No siempre coincide con la que se habría reportado sin diferir, y ahí está la ganancia: en `x -> ((y) -> z)` el diferimiento descarta el `y` —resultó ser un parámetro— y deja el `z`, que es la captura real.

La gramática ya distingue los tres usos de un **idMetVar** —acceso a variable, llamada, y campo después de un **.**—, así que se sabe cuál hay que chequear contra los parámetros y cuál no: `x -> x.campo` y `x -> C.metodo(x)` son válidos, `x -> x + n` no.

Se hace en el analizador y no se posterga al semántico porque el logro es de esta etapa, porque el costo es un booleano y un conjunto de cadenas —no hay tabla de símbolos ni alcances anidados—, y porque no toca ninguna producción: es un chequeo sobre lo ya parseado, del mismo tipo que la regla que resuelve el *dangling else*, y Clase 4 slide 49 lo autoriza expresamente.

Lo que sí queda para el analizador semántico es todo lo que exija resolver un nombre contra una declaración: que exista, que el tipo cierre, que la aridad coincida.

Pendiente declarado para la Etapa 3. La cátedra admite no controlar en esta etapa si un **this** —explícito o implícito— está dentro o fuera de una lambda, porque con la tabla de símbolos del análisis semántico el control es mucho más directo. Acá está implementado igual, con el alcance que describe esta decisión, y se deja como está. Lo que queda para la Etapa 3 es el único caso que este analizador no puede decidir: la llamada sin punto `x -> ayuda(x)`, que es un **this** implícito si `ayuda` es un método de instancia y no lo es si es **static**. Distinguirlos exige resolver el nombre contra su declaración, así que acá se acepta.

Vale además dejar dicho por qué diferirlo no habría simplificado nada: **el control del this implícito no es separable del de la captura**. En el cuerpo de una lambda, `x -> campo` —un atributo, o sea **this** implícito— y `x -> n` —una local o un parámetro de afuera, o sea una captura— son *la misma forma*: un **idMetVar** pelado que no está entre los parámetros. Un solo `if` rechaza los dos, y separarlos exigiría resolver el nombre contra su declaración, que es justamente lo que el sintáctico no hace. La consecuencia práctica es que quitar el control del **this** implícito significaría quitar también el de la captura, que el logro exige por separado; y que todo el estado que este chequeo necesita —el alcance, el conjunto de parámetros y el diferimiento— se sostiene por la cláusula de la captura, no por la del **this**.

6.8 Inicializacion de Arreglos E2

Después del **[** de una creación de arreglo, o viene una expresión de tamaño o viene el **]** seguido del inicializador. Un token de desempate.

`<CreacionDeArreglo>` ::= **[** `<RestoCreacionDeArreglo>`

`<RestoCreacionDeArreglo>` ::= `<Expresion>` **]** `<DimensionesConTamanioOpcional>`

`<RestoCreacionDeArreglo>` ::= **]** **{** `<ListaExpsOpcional>` **}**

El logro dice «como en Java» y la única restricción que enuncia es *dónde* —dentro de un **new**—, no cuántas dimensiones. Como Java admite `new int[][]{{1,2},{3,4}}`, el inicializador es **anidable**: un elemento puede ser una expresión o, a su vez, otro inicializador.

`<DimensionesVaciasOpcional>` ::= **[ ]** `<DimensionesVaciasOpcional>` | ϵ

`<InicializadorDeArreglo>` ::= **{** `<ListaElementosOpcional>` **}**

`<ElementoInicializador>` ::= `<Expresion>` | `<InicializadorDeArreglo>`

Primeros(`<Expresion>`) no contiene **{**, así que las dos alternativas de `<ElementoInicializador>` son disjuntas. La restricción de que sólo valga como parte de un **new** queda impuesta por la gramática: `<CreacionDeArreglo>` sólo es

alcanzable desde `<Creacion>`.

6.9 Genericidad Avanzada E2

Dos cosas distintas. El **anidamiento** sale de una sola recursión:

`<ArgumentoDeTipo>` ::= **idGen** | **idClase** `<TipoGenericoOpcional>`

Con el logro apagado la segunda alternativa es **idClase** a secas. (`List<List<Persona>>` cierra con dos `>` separados: el léxico no define ningún token `>>`, así que la regla de *maximal munch* entrega dos `>` y no hay nada que resolver del lado sintáctico.)

El **diamante** exige más cuidado, porque el enunciado dice que «sólo puede utilizarse al instanciar una Clase Genérica». Se impone **por gramática**, separando el genérico en posición de tipo del genérico bajo **new**:

`<TipoGenericoOpcional>` ::= `<` `<ArgumentoDeTipo>` `>` | ϵ

`<CreacionDesdeIdClase>` ::= `<` `<GenericoDeCreacion>` | `<CreacionDeArreglo>` | `<ArgsActuales>`

`<GenericoDeCreacion>` ::= `>` `<ArgsActuales>`

`<GenericoDeCreacion>` ::= `<ArgumentoDeTipo>` `>` `<ArregloOInstanciaDeClase>`

La primera alternativa de `<GenericoDeCreacion>` *es* el diamante, y va sólo hacia `<ArgsActuales>`. Resultado:

Entrada	Veredicto
`List<Persona> x;`	acepta
`List<List<Persona>> x;`	acepta
`new List<>()`	acepta
`new List<Persona>[3]`	acepta
`List<> x;`	**rechaza** (diamante en posición de tipo)
`new List<>[3]`	**rechaza** (diamante con arreglo)

Decisión — compartir un no terminal puede volverse incorrecto más adelante

En la gramática original, `<TipoGenericoOpcional>` se usa desde `<TipoReferencia>` y desde `<LlamadaConstructor>`, y compartirlo es correcto: son la misma construcción. **Deja de serlo con este logro**, porque el diamante vale en una posición y no en la otra. Compartiéndolo habría que aceptar `List<> x;` y rechazarlo después en el análisis semántico. Separarlos cuesta un no terminal y mueve la restricción al lugar donde se puede verificar sin conocer el significado de nada.

6.10 Floats (*carry-over* de la Etapa 1)

`<TipoPrimitivo>` ::= **boolean** | **char** | **int** | **float**

`<Primitivo>` ::= **true** | **false** | **intLiteral** | **charLiteral** | **floatLiteral** | **null**

Dos alternativas nuevas, cero conflictos. Sin ellas, el logro de floats de la Etapa 1 queda inutilizable en la Etapa 2: el léxico emite **float** y **floatLiteral** y el analizador sintáctico los rechazaría.

6.11 Impacto sobre el analizador léxico

Cuatro de los logros exigen elementos léxicos que **no figuran en *Aspectos Léxicos de* MiniJava**: la lista de palabras clave de §2.1 llega hasta **false** y la de operadores de §2.4.2 hasta ++ y --.

Elemento	Logro	Estado en la Etapa 1
for	Fors! E2	se reconoce como **idMetVar**
private	Visibilidad Mejorada E2	se reconoce como **idMetVar**
?	Operador Ternario E2	es un error léxico
->	MiniLambdas E2	se reconoce como - seguido de >

Se incorporan al analizador léxico **condicionados al interruptor de su logro**: con el logro apagado, **?** vuelve a ser un error léxico y **for** y **private** vuelven a ser identificadores, exactamente como especifica la cátedra. De ese modo el comportamiento léxico base de la Etapa 1 se conserva íntegro y verificable, que es lo que exige la condición de atomicidad de la sección 6.1. Los tokens **:**, ++ y --, en cambio, ya existían.

7 Gramática final

La gramática completa resultante: núcleo corregido más los nueve logros de la Etapa 2 y el *carry-over* de floats. Las producciones marcadas con *[sólo con `logro`]* existen únicamente cuando ese interruptor está prendido; las marcadas *[sólo sin `logro`]*, únicamente cuando está apagado. Las que no llevan marca existen siempre.

Decisión — este listado se genera, no se transcribe

El texto que sigue se produce automáticamente a partir del mismo archivo que verifica la herramienta de la sección 11, de modo que el informe y lo que se comprueba no puedan divergir. Cualquier corrección se hace sobre la gramática y se regenera el informe, nunca al revés.

<Inicial> ::= <ListaClases> **eof**

<ListaClases> ::= <Clase> <ListaClases> | <Interfaz> <ListaClases> | ϵ

<Clase> ::= **class idClase** <GenericidadOpcional> <HerenciaOpcional> **{** <ListaMiembros> **}**

<Interfaz> ::= **interface idClase** <GenericidadOpcional> <ExtensionOpcional> **{** <ListaMetodosInterfaz> **}**

Una Clase o Interfaz sigue limitada a UN único parámetro de tipo, también con el logro de genericidad avanzada.

<GenericidadOpcional> ::= **<** **idGen** **>** | ϵ

<HerenciaOpcional> ::= **extends** <TipoReferencia> | **implements** <TipoReferencia> | ϵ

<ExtensionOpcional> ::= **extends** <TipoReferencia> | ϵ

<ListaMiembros> ::= <Miembro> <ListaMiembros> | ϵ

<ListaMetodosInterfaz> ::= <MetodoInterfaz> <ListaMetodosInterfaz> | ϵ

La visibilidad se factoriza al frente de los tres tipos de miembro. Eso hace que el `public` del `<Constructor>` original quede absorbido por `<VisibilidadOpcional>`, y que `<Constructor>`

pase a competir con `<Tipo>` por el token idClase: `C(){}` (constructor) vs `C x;` (atributo de tipo C).

`<Miembro>` ::= `<VisibilidadOpcional>` `<MiembroSinVisibilidad>` [+ visibilidad]

`<Miembro>` ::= `<MiembroSinVisibilidad>` [- visibilidad]

`<Miembro>` ::= `<Constructor>` [- visibilidad]

`<VisibilidadOpcional>` ::= **public** | **private** | ϵ [+ visibilidad]

Sin el logro, el `public` sólo aparece en el constructor y es lo que lo distingue del resto de los miembros: es la regla original de la cátedra.

`<Constructor>` ::= **public idClase** `<CuerpoDeMetodo>` [- visibilidad]

`<MiembroSinVisibilidad>` ::= **static** `<TipoMetodo>` `<Metodo>`

`<MiembroSinVisibilidad>` ::= **void** `<Metodo>`

`<MiembroSinVisibilidad>` ::= `<TipoPrimitivo>` `<DimensionesOpcionales>` **idMetVar** `<AtributoOMetodo>`

`<MiembroSinVisibilidad>` ::= **idGen** `<DimensionesOpcionales>` **idMetVar** `<AtributoOMetodo>`

`<MiembroSinVisibilidad>` ::= **idClase** `<MiembroDesdeIdClase>`

El NT insertado que decide entre constructor y miembro con tipo de referencia. Con visibilidad el constructor ya no lleva `public` propio, así que compite con el atributo/método de tipo referencia por el token idClase.

`<MiembroDesdeIdClase>` ::= `<CuerpoDeMetodo>` [+ visibilidad]

`<MiembroDesdeIdClase>` ::= `<TipoGenericoOpcional>` `<DimensionesOpcionales>` **idMetVar** `<AtributoOMetodo>`

El NT insertado que decide entre atributo y método.

`<AtributoOMetodo>` ::= `<InicializacionDeAtributo>` **;**

`<AtributoOMetodo>` ::= `<CuerpoDeMetodo>`

NT dedicado: la inicialización de un ATRIBUTO es un logro distinto que la de una variable local, y tienen que poder prenderse por separado.

`<InicializacionDeAtributo>` ::= **=** `<ExpresionCondicionalOLambda>` [+ atrinit]

`<InicializacionDeAtributo>` ::= ϵ

`<Metodo>` sobrevive como no terminal propio y lo comparten la rama static y la rama void, así que el semántico tiene UN solo lugar donde construir el nodo de método para esas dos formas. La tercera forma (tipo de referencia) ya consumió el idMetVar al factorizar contra el atributo, y llega por AtributoOMetodo.

`<Metodo>` ::= **idMetVar** `<CuerpoDeMetodo>`

Se comparte a propósito: es la misma construcción en método, método static y constructor.

`<CuerpoDeMetodo>` ::= `<ArgsFormales>` `<Bloque>`

Los métodos de interfaz admiten `public` pero no `private`, que es lo que hace Java: verificado con javac, un método de interfaz `private` exige cuerpo («missing method body, or declare abstract»), y MiniJava no tiene forma de escribir ese cuerpo —`<MetodoInterfaz>` termina en `;` en la especificación original y ningún logro lo cambia—. O sea que un `private` acá sería siempre la forma que Java rechaza.

`<MetodoInterfaz>` ::= `<PublicOpcional>` `<TipoMetodo>` **idMetVar** `<ArgsFormales>` **;**

<PublicOpcional> ::= **public** [+ visibilidad]

<PublicOpcional> ::= ϵ

<TipoMetodo> ::= <Tipo> | **void**

<Tipo> ::= <TipoBase> <DimensionesOpcionales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | **idGen**

<DimensionesOpcionales> ::= **[ ]** <DimensionesOpcionales> | ϵ

<TipoReferencia> ::= **idClase** <TipoGenericoOpcional>

`float` es token del léxico desde la Etapa 1 pero no estaba en la gramática; sin esta alternativa el logro de Etapa 1 queda inutilizable.

<TipoPrimitivo> ::= **boolean** | **char** | **int**

<TipoPrimitivo> ::= **float** [+ float]

En POSICIÓN DE TIPO no se admite el diamante: `List<> x;` es inválido.

<TipoGenericoOpcional> ::= **<** <ArgumentoDeTipo> **>** | ϵ

la recursión por <TipoGenericoOpcional> es la que habilita el anidamiento `List<List<Persona>>`. Sin el logro: `ArgumentoDeTipo -> idGen | idClase`.

<ArgumentoDeTipo> ::= **idGen**

<ArgumentoDeTipo> ::= **idClase** <TipoGenericoOpcional> [+ generico]

<ArgumentoDeTipo> ::= **idClase** [- generico]

<ArgsFormales> ::= **(** <ListaArgsFormalesOpcional> **)**

<ListaArgsFormalesOpcional> ::= <ListaArgsFormales> | ϵ

<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>

<RestoListaArgsFormales> ::= **,** <ArgFormal> <RestoListaArgsFormales> | ϵ

<ArgFormal> ::= <Tipo> **idMetVar**

Con el logro de variables locales clásicas una declaración local puede empezar con idClase (`C x;`), y una sentencia-expresión también (`C.m();`). Se factoriza ese idClase y el desempate lo hace el token siguiente: `.` es expresión, cualquier otro es declaración. La rama de expresión reusa <ColaDeExpresionDesdePrimario> para no duplicar la jerarquía de expresiones.

<Bloque> ::= **{** <ListaSentencias> **}**

<ListaSentencias> ::= <Sentencia> <ListaSentencias> | ϵ

<Sentencia> ::= **;**

<Sentencia> ::= <VarLocal> **;**

<Sentencia> ::= <Return> **;**

<Sentencia> ::= <If>

<Sentencia> ::= <While>

<Sentencia> ::= <Bloque>

<Sentencia> ::= <For> [+ for]

Sin variables locales clásicas, una sentencia-expresión puede empezar con idClase sin competir con nada: es la regla original.

<Sentencia> ::= <SentenciaExpresion> **;** [- varclasica]

<SentenciaExpresion> ::= <ExpresionSinLambda> [- varclasica]

Con el logro, `C x;` y `C.m();` compiten por idClase y hay que factorizarlo.

<Sentencia> ::= <ExpresionSinIdClase> **;** [+ varclasica]

<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales>
<ListaDeclaradores> ; [+varclasica]

<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;
[+varclasica]

<Sentencia> ::= idClase <SentenciaDesdeIdClase> [+varclasica]

<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales>
<ListaDeclaradores> ; [+varclasica]

<SentenciaDesdeIdClase> ::= . idMetVar <ArgsActuales>
<ColaDeExpresionDesdePrimario> ; [+varclasica]

<VarLocal> ::= var idMetVar = <ExpresionCondicionalOLambda>

Varias variables por sentencia, cada una con inicialización opcional: `int x,y,z = 10;`

<ListaDeclaradores> ::= idMetVar <InicializacionDeDeclarador>
<RestoListaDeclaradores> [+varclasica]

<RestoListaDeclaradores> ::= , idMetVar <InicializacionDeDeclarador>
<RestoListaDeclaradores> [+varclasica]

<RestoListaDeclaradores> ::= ϵ [+varclasica]

<InicializacionDeDeclarador> ::= = <ExpresionCondicionalOLambda> | ϵ
[+varclasica]

<Return> ::= return <ExpresionOpcional>

<ExpresionOpcional> ::= <Expresion> | ϵ

<If> ::= if (<Expresion>) <Sentencia> <ElseOpcional>

<ElseOpcional> ::= else <Sentencia> | ϵ

<While> ::= while (<Expresion>) <Sentencia>

Fors! E2 Una sola sentencia/expresión por separador. El : distingue el for de iteradores del estándar, y aparece justo después del declarador, así que el desempate es de un solo token.

<For> ::= for (<EncabezadoFor>) <Sentencia> [+for]

<EncabezadoFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional> [+for]

<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar> [+for]

<EncabezadoFor> ::= <TipoPrimitivo> <DimensionesOpcionales> idMetVar
<EncabezadoForTrasDeclarador> [+for]

<EncabezadoFor> ::= idGen <DimensionesOpcionales> idMetVar
<EncabezadoForTrasDeclarador> [+for]

<EncabezadoFor> ::= idClase <EncabezadoForDesdeIdClase> [+for]

<EncabezadoFor> ::= <ExpresionSinIdClase> <ClausulasFinalesFor> [+for]

<EncabezadoForDesdeIdClase> ::= <TipoGenericoOpcional>
<DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador> [+for]

<EncabezadoForDesdeIdClase> ::= . idMetVar <ArgsActuales>
<ColaDeExpresionDesdePrimario> <ClausulasFinalesFor> [+for]

El : distingue el for de iteradores del estándar, y está a un solo token.

<EncabezadoForTrasVar> ::= : <Expresion> [+for]

<EncabezadoForTrasVar> ::= = <ExpresionCondicionalOLambda>
<ClausulasFinalesFor> [+for]

<EncabezadoForTrasDeclarador> ::= : <Expresion> [+for]

<EncabezadoForTrasDeclarador> ::= <InicializacionDeDeclaradorFor>
<ClausulasFinalesFor> [+ for]

<InicializacionDeDeclaradorFor> ::= = <ExpresionCondicionalOLambda> | ϵ
[+ for]

Una sola expresión por separador, como pide el logro.

<ClausulasFinalesFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional> [+ for]

Las expresiones se organizan en capas, de menor a mayor precedencia: <Expresion> es la asignación (no asociativa, como en la gramática original); <ExpresionCondicional> el ternario; <ExpresionCompuesta> los operadores binarios, en un solo nivel como en el original; <ExpresionBasica> los operadores unarios prefijos; y <Operando> un literal, o una referencia con posfijo opcional. La lambda NO es una capa de esta jerarquía ni un <Primario>: es una alternativa del nivel MÁS ALTO, igual que en el JLS, que define `Expression: LambdaExpression | AssignmentExpression`. Por eso `->` no tiene precedencia contra ningún operador —no compite con ellos, vive arriba de todos—, y `1 + x -> x`, `- x -> x` y `a == x -> x` son inválidos, como en Java. Las dos cabezas que una lambda comparte con una expresión normal, `idMetVar` y `(`, se factorizan en <ExpresionDesdePrimarioOLambda>. Todo lado derecho de un `=` —el de la asignación y el de los cinco inicializadores— es una <ExpresionCondicionalOLambda> y no una <Expresion>. Eso preserva lo que la regla original conseguía usando <ExpresionCompuesta>: que `a = b = c` no sea válido. Y como con los logros de ternario y de lambda apagados <ExpresionCondicionalOLambda> es un alias de <ExpresionCompuesta>, el lenguaje base queda idéntico al de la cátedra.

<Expresion> ::= <ExpresionCondicionalOLambda> <RestoExpresion>

La <Expresion> de la gramática original: la que NO admite una lambda en la raíz. Se usa en los dos lugares donde Java tampoco la admite: adentro de una expresión parentizada que después va a ser operando, y como sentencia- expresión (`x -> x;` es «not a statement» en Java).

<ExpresionSinLambda> ::= <ExpresionCondicional> <RestoExpresion>

El lado derecho es ExpresionCondicionalOLambda y NO Expresion: la gramática original pone <ExpresionCompuesta> ahí, o sea que `a = b = c` NO es válido. Admite lambda porque el JLS define `Assignment: LeftHandSide = Expression`.

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicionalOLambda> | ϵ

<OperadorAsignacion> ::= =

El nivel del JLS `LambdaExpression | ConditionalExpression`: admite lambda, no admite `=` en la raíz. Es el operando de los seis `=` y el tercer operando del ternario. Las cabezas que NO pueden empezar una lambda delegan en la jerarquía de siempre; las dos que sí —`idMetVar` y `(`— se factorizan. La cola se comparte con esa jerarquía, así que no hay lógica duplicada: sólo las cabezas.

<ExpresionCondicionalOLambda> ::= <OperadorUnario> <Operando>
<ColaTrasOperandoSinAsignacion>

<ExpresionCondicionalOLambda> ::= <Primitivo>
<ColaTrasOperandoSinAsignacion>

<ExpresionCondicionalOLambda> ::= <ExpresionDesdePrimarioOLambda>

<ExpresionDesdePrimarioOLambda> ::= idClase . idMetVar <ArgsActuales>
<ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= this <ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= **stringLiteral**
<ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= <Creacion>
<ColaDesdePrimarioSinAsignacion>

<ExpresionDesdePrimarioOLambda> ::= **idMetVar** <AccesoLlamadaOLambda>

<ExpresionDesdePrimarioOLambda> ::= <ParentizadaOLambda>

<ExpresionCondicional> ::= <ExpresionCompuesta> <RestoCondicional>

Con el logro apagado <RestoCondicional> queda en solo ϵ, o sea que <ExpresionCondicional> se vuelve un alias de <ExpresionCompuesta> y la capa desaparece sin dejar rastro. Asociativo a derecha, como en Java. El operando del MEDIO es una <Expresion> y el de la derecha una <ExpresionCondicionalOLambda>, tal cual el JLS 15.25: en Java `a ? b = 1 : c` es válido, `a ? b : c = d` no lo es, y `a ? b : x -> x` sí —la producción del JLS es `? Expression : LambdaExpression`—. No rompe LL(1) porque Primeros(Expresion) no contiene `:`.

<RestoCondicional> ::= **?** <Expresion> **:** <ExpresionCondicionalOLambda>
[+ `ternario`]

<RestoCondicional> ::= ϵ

<ExpresionCompuesta> ::= <ExpresionBasica> <RestoExpresionCompuesta>

<RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>
<RestoExpresionCompuesta> | ϵ

<OperadorBinario> ::= || | **&&** | == | **!=** | < | > | <= | >= | + | - | * | / | **%**

<ExpresionBasica> ::= <OperadorUnario> <Operando>

<ExpresionBasica> ::= <Operando>

<OperadorUnario> ::= + | - | **!**

<Operando> ::= <Primitivo>

<Operando> ::= <Referencia> <PosfijoOpcional>

Va sobre <Referencia> y no sobre <Operando>: `5++` no tiene sentido.

<PosfijoOpcional> ::= ++ [+ `posfijo`]

<PosfijoOpcional> ::= -- [+ `posfijo`]

<PosfijoOpcional> ::= ϵ

floatLiteral

<Primitivo> ::= **true** | **false** | **intLiteral** | **charLiteral** | **null**

<Primitivo> ::= **floatLiteral** [+ `float`]

Expresión que no puede empezar con idClase. Se usa donde una expresión compite con una declaración de variable local (sentencia y encabezado del for). Sólo duplica las *cabezas*: todas las colas se comparten con la jerarquía normal, así que no hay lógica repetida. NO admite lambda en la raíz, por la misma razón que <ExpresionSinLambda>: los dos contextos donde se usa son sentencia-expresión e inicialización del for, y en Java los dos exigen un StatementExpression, que nunca es una lambda.

<ExpresionSinIdClase> ::= <OperadorUnario> <Operando> <ColaTrasOperando>

<ExpresionSinIdClase> ::= <Primitivo> <ColaTrasOperando>

<ExpresionSinIdClase> ::= <PrimarioSinIdClase>
<ColaDeExpresionDesdePrimario>

La cola se parte en dos para que el nivel <ExpresionCondicionalOLambda> pueda reusarla sin arrastrar el =, que ahí no corresponde.

<ColaTrasOperando> ::= <ColaTrasOperandoSinAsignacion> <RestoExpresion>

<ColaTrasOperandoSinAsignacion> ::= <RestoExpresionCompuesta> <RestoCondicional>

<ColaDeExpresionDesdePrimario> ::= <RestoReferencia> <PosfijoOpcional> <ColaTrasOperando>

<ColaDesdePrimarioSinAsignacion> ::= <RestoReferencia> <PosfijoOpcional> <ColaTrasOperandoSinAsignacion>

Ningún <Primario> puede ser una lambda: ésa es exactamente la corrección de precedencia. Un <Primario> es un operando, y una lambda no puede serlo.

<Referencia> ::= <Primario> <RestoReferencia>

<RestoReferencia> ::= <Encadenado> <RestoReferencia>

<RestoReferencia> ::= <AccesoArreglo> <RestoReferencia>

<RestoReferencia> ::= ϵ

<Encadenado> ::= . idMetVar <EncadenadoAtributoOMetodo>

Dedicado a este contexto: vacío = era <VarEncadenada>, con argumentos = era <MetodoEncadenado>.

<EncadenadoAtributoOMetodo> ::= <ArgsActuales> | ϵ

<AccesoArreglo> ::= [<Expresion>]

<Primario> ::= <LlamadaMetodoEstatico>

<Primario> ::= <PrimarioSinIdClase>

<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>

<PrimarioSinIdClase> ::= this

<PrimarioSinIdClase> ::= stringLiteral

<PrimarioSinIdClase> ::= <Parentizada>

<PrimarioSinIdClase> ::= <AccesoVarOLlamada>

<PrimarioSinIdClase> ::= <Creacion>

El contenido es una <ExpresionSinLambda>: `(x -> x)` como OPERANDO no existe en Java —`(x -> x) + 1` y `(x -> x).m()` son «lambda expression not expected here»—. La lambda parentizada sí es válida cuando ella misma es toda la expresión, y ese caso lo cubre <ParentizadaOLambda>.

<Parentizada> ::= (<ExpresionSinLambda>)

Dedicado a este contexto: vacío = era <AccesoVar>, con argumentos = era <LlamadaMetodo>.

<AccesoVarOLlamada> ::= idMetVar <AccesoLlamada>

<AccesoLlamada> ::= <ArgsActuales> | ϵ

MiniLambdas E2 Las dos cabezas ambiguas, resueltas en el nivel más alto de la jerarquía. `(` abre tanto una expresión parentizada como la lista de parámetros de una lambda. Parece necesitar lookahead ilimitado (`(a,b,c) ->` vs `(a+b)`), pero NO lo necesita: - `( )` sólo puede ser una lambda de cero parámetros; - `( e , ...` sólo puede ser una lambda: MiniJava no tiene operador coma; - `( e )` se decide con el token que sigue al `)`: `->` es lambda, cualquier otro es una expresión parentizada. Con eso alcanza un token de lookahead en cada punto. Con flecha = lambda de un parámetro sin paréntesis; si no, es el <AccesoVar> o la <LlamadaMetodo> de siempre, con la cola de expresión que le corresponde.

<AccesoLlamadaOLambda> ::= -> <CuerpoLambda> [+ lambda]

<AccesoLlamadaOLambda> ::= <AccesoLlamada>
<ColaDesdePrimarioSinAsignacion>

<ParentizadaOLambda> ::= (<ContenidoDeParentesis>

<ContenidoDeParentesis> ::=) -> <CuerpoLambda> [+ lambda]

<ContenidoDeParentesis> ::= <Expresion> <RestoDeParentesis>

<RestoDeParentesis> ::=) <FinDeParentizadaOLambda>

<RestoDeParentesis> ::= , <ListaParametrosLambda>) -> <CuerpoLambda>
[+ lambda]

<FinDeParentizadaOLambda> ::= -> <CuerpoLambda> [+ lambda]

<FinDeParentizadaOLambda> ::= <ColaDesdePrimarioSinAsignacion>

<ListaParametrosLambda> ::= idMetVar <RestoListaParametrosLambda>
[+ lambda]

<RestoListaParametrosLambda> ::= , idMetVar <RestoListaParametrosLambda> |
ϵ [+ lambda]

Una única expresión, sin llaves.

<CuerpoLambda> ::= <Expresion> [+ lambda]

<Creacion> ::= new <ArregloOInstancia>

<ArregloOInstancia> ::= <TipoPrimitivo> <CreacionDeArreglo>

<ArregloOInstancia> ::= idGen <CreacionDeArreglo>

<ArregloOInstancia> ::= idClase <CreacionDesdeIdClase>

<CreacionDesdeIdClase> ::= < <GenericoDeCreacion>

<CreacionDesdeIdClase> ::= <CreacionDeArreglo>

<CreacionDesdeIdClase> ::= <ArgsActuales>

el diamante <> sólo se admite acá, en la rama que va a ArgsActuales: `new C<>()` es válido y `new C<>[3]` no. La restricción del logro queda impuesta por la gramática, no delegada al semántico.

<GenericoDeCreacion> ::= > <ArgsActuales> [+ generico]

<GenericoDeCreacion> ::= <ArgumentoDeTipo> > <ArregloOInstanciaDeClase>

<ArregloOInstanciaDeClase> ::= <CreacionDeArreglo>

<ArregloOInstanciaDeClase> ::= <ArgsActuales>

`new int[]{1,2,3}`. Sin el logro: CreacionDeArreglo -> [Expresion] DimensionesConTamanioOpcional

<CreacionDeArreglo> ::= [<RestoCreacionDeArreglo>

<RestoCreacionDeArreglo> ::= <Expresion>] <DimensionesConTamanioOpcional>

El logro dice «como en Java» y sólo restringe DÓNDE (dentro de un `new`), no cuántas dimensiones. Java admite `new int[][]{{1,2},{3,4}}`, así que el inicializador es anidable: un elemento puede ser una expresión o, a su vez, otro inicializador.

<RestoCreacionDeArreglo> ::=] <DimensionesVaciasOpcional>
<InicializadorDeArreglo> [+ arrinit]

<DimensionesVaciasOpcional> ::= [ ] <DimensionesVaciasOpcional> | ϵ
[+ arrinit]

<InicializadorDeArreglo> ::= { <ListaElementosOpcional> } [+ arrinit]

<ListaElementosOpcional> ::= <ListaElementos> | ϵ [+ arrinit]

<ListaElementos> ::= <ElementoInicializador> <RestoListaElementos>
[+arrinit]

<RestoListaElementos> ::= , <ElementoInicializador> <RestoListaElementos>
| ϵ [+arrinit]

<ElementoInicializador> ::= <Expresion> | <InicializadorDeArreglo>
[+arrinit]

<DimensionesConTamanio> ::= [<Expresion>] <DimensionesConTamanioOpcional>

<DimensionesConTamanioOpcional> ::= <DimensionesConTamanio> | ϵ

Sección propia y no parte de la creación: la lista de argumentos de una llamada la usan por igual el `new`, el encadenado, la llamada estática y el encabezado del for.

<ArgsActuales> ::= (<ListaExpsOpcional>)

<ListaExpsOpcional> ::= <ListaExps> | ϵ

<ListaExps> ::= <Expresion> <RestoListaExps>

<RestoListaExps> ::= , <Expresion> <RestoListaExps> | ϵ

8 Cómo compilar y correr

El proyecto se construye con Maven y apunta a Java 25. Desde la carpeta `compilador/`, que es donde está el `pom.xml`:

```
$ mvn package          # compila, corre la suite y arma target/Compilador.jar
$ mvn verify           # ademas verifica los umbrales de cobertura
$ mvn test             # solo la suite
```

El jar resultante es ejecutable y recibe el fuente como único argumento, tal cual pide Pautas slide 28. Acepta cualquier extensión:

```
$ java -jar target/Compilador.jar programa1.java
Compilacion Exitosa

[SinErrores]
```

Dos opciones más, que no hacen falta para la corrección normal pero permiten ejercitar el compilador entero:

```
# La Etapa 1 sigue siendo ejecutable sola: vuelca la lista de tokens.
$ java -jar target/Compilador.jar --etapa=1 programa1.java

# Prender y apagar logros. Sin --logros manda la etapa: la Etapa 2 los trae todos.
$ java -jar target/Compilador.jar --logros=ninguno programa1.java
$ java -jar target/Compilador.jar --logros=ternario,lambda programa1.java
```

Una opción o un nombre de logro desconocidos se informan por salida estándar y **no** abortan: los testers de la cátedra invocan `main()` dentro del mismo proceso, y una excepción o un `System.exit()` matarían la corrida entera. Por el mismo motivo el compilador no llama nunca a `System.exit()`.

9 El analizador sintáctico

9.1 La técnica

Analizador **Descendente Recursivo Predictivo LL(1)**, con el **Esquema Simple** de Clase 5: un método por no terminal, `match` para los terminales, y la producción se elige mirando si el token actual está en los Primeros del cuerpo de alguna de ellas. **Sólo Primeros**: los Siguientes no aparecen en ninguna línea del compilador.

Los métodos tienen tres formas, según cuántas producciones tenga el no terminal y si alguna deriva a vacío:

Forma del NT	Traducción
Una sola producción	El cuerpo, sin condición.
Varias, ninguna anulable	Una cadena de `if/else if` sobre los Primeros de cada cuerpo, y un `else` que lanza el error (slide 36).
Varias, alguna anulable	Lo mismo, pero el `else` **no hace nada**: que el token no coincida con ninguno significa que el no terminal derivó a vacío, y quien llamó decidirá si eso es un error (slides 52–54).

Ese `else` vacío **se escribe igual**, con la producción ε citada al lado, en vez de omitirlo dejando que el `if` caiga solo. El analizador hace lo mismo en los dos casos, pero omitirlo borra del código el único lugar donde está dicho «acá el no terminal derivó a vacío» —que es exactamente el punto donde el análisis semántico de la Etapa 3 tiene que colgar la traducción de esa producción—.

La única libertad que se toma respecto del esquema es traducir la **recursión a derecha con un bucle** en vez de con una llamada: deriva exactamente la misma cadena y no hace depender de la pila de la JVM una cadena larga de operadores o una lista larga de argumentos.

9.2 Las clases

El analizador tiene 111 no terminales. Escribirlos en una sola clase daba 1176 líneas y 89 métodos, así que están repartidos en **una clase por sección de la gramática**: el archivo de código y la sección del `.g` se leen uno al lado del otro.

Clase	Qué traduce
`AnalizadorSintactico`	El arranque y el contrato de uso: carga el primer token, llama al símbolo inicial y devuelve la lista de errores.
`EntradaDeTokens`	El lookahead y **el único punto que consume entrada**.
`Sintaxis`	Cablea las once secciones. Existe porque la gramática tiene recursión mutua entre ellas —una sentencia contiene expresiones, una expresión contiene lambdas, el cuerpo de una lambda vuelve a ser una expresión— y por lo tanto no se pueden inyectar unas en otras por constructor sin un ciclo.
`ParteDelAnalizador`	La base de las once: delega `match`, `actual`, `estaEn` y `hayLogro` para que cada método se lea como en la slide 57.
`AnalizadorDe{Clases, Miembros, Tipos, Sentencias, Fors, Expresiones, Referencias, Lambdas, Creacion, Argumentos}`	Una sección de la gramática cada una, entre 99 y 173 líneas.
`primeros/PrimerosDeX`	Una constante por producción, **generada** desde la gramática: 220 en total, divididas por la misma sección.
`RestriccionesDeLambda`	Los chequeos del logro MiniLambdas que la gramática no expresa.
`recuperacion/ {RecuperadorDePanico, ContextoDeSincronizacion}`	El logro Modo Pánico.
`errores/ExcepcionSintactica`	El único error que produce el analizador.

Que el código y la gramática no se separen no depende de la disciplina: cada método cita arriba las producciones que traduce, y `auditar_analizador.py` falla si una producción no la implementa nadie o si un comentario quedó viejo (sección 11).

9.3 Los tipos de error que detecta

Todo error que emite el analizador informa **las dos mitades**: qué se encontró —el lexema real, entre comillas— y qué se esperaba en su lugar. No es una elección de redacción: son literalmente los dos datos con los que el Esquema Simple decide, el token actual y el conjunto de Primeros de la producción, así que el mensaje es la traducción directa de la condición que falló.

El Esquema Simple tiene **exactamente dos** puntos donde puede fallar, y no hay un tercero:

1. **Un `match` que no coincide.** Se esperaba un terminal concreto y llegó otro. *«se esperaba `;` se encontro `"void"`».*
2. **Un no terminal NO anulable cuyo token actual no está en los Primeros de ninguna de sus producciones.** *«se esperaba `this`, `new`, ... o `(` se encontro `";"`».* Cuando el no terminal *sí* es anulable esto no es un error: derivó a vacío y lo resolverá quien llamó.

En el primer caso «lo que se esperaba» es un terminal único; en el segundo, la lista completa de los terminales que podrían haber seguido, que es exactamente el conjunto de Primeros consultado.

A esos se suman los **cuatro chequeos del logro MiniLambdas** que una gramática libre de contexto no puede expresar (sección 12): el `this` explícito dentro de una lambda, la captura de una variable que no es parámetro, que un parámetro sea un identificador pelado y no se repita, y que una lambda parentizada no lleve cola de expresión. Rechazan con el mismo código `[Error:lexema|línea]` y, como ahí «lo esperado» no es un conjunto de tokens sino una regla, declaran el motivo en prosa —*«no se permite this dentro de una lambda»*, *«una lambda no puede ser operando de .»*— sin perder la mitad de «qué se encontró».

Y una aclaración que conviene dejar escrita: los errores **léxicos** los sigue detectando y reportando el analizador de la Etapa 1, con su propio formato. El sintáctico no los toca.

9.4 El formato del reporte

Pautas slide 25: un mensaje que nombre el lexema y la línea, un renglón en blanco y el código. El mensaje abre con lo esperado y cierra con lo encontrado, en ese orden.

```
Error Sintactico en linea 3: se esperaba idClase se encontro "clasesita"

[Error:clasesita|3]
```

El lexema del token EOF es **$**, como fija Pautas slide 26. Con el logro *Modo Pánico* prendido el reporte entero se repite por cada error encontrado, en orden, que es el mismo formato con el que la Etapa 1 reporta varios errores léxicos.

10 Logros perseguidos

Se persiguen **todos los logros de la etapa**: los diez que son implementables por código —más el *carry-over* de *Floats* de la Etapa 1— y los tres que no lo son. Cada uno de los implementables por código es un **interruptor independiente**: el compilador se puede ejecutar y testear con 0, 1 o N logros en cualquier combinación, y la de cero logros reproduce exactamente la especificación base de la cátedra. Eso está verificado sobre las 1024 configuraciones.

Logro	`--logros=`	Dónde está
MiniLambdas E2	`lambda`	§12 y las seis producciones del bloque de lambdas. Requiere el token `->`.
Variables locales Clásicas E2	`varclasica`	La factorización de **idClase** en `<Sentencia>`.
Visibilidad Mejorada E2	`visibilidad`	`<VisibilidadOpcional>` al frente de los tres miembros. Requiere el token **private**.
Fors! E2	`for`	El bloque `<For>` entero, estándar y de iteradores. Requiere el token **for**.
Operador Ternario E2	`ternario`	La capa `<ExpresionCondicional>`. Requiere el token **?**, que sin el logro es un error léxico.
Operadores Posifijos E2	`posfijo`	`<PosfijoOpcional>`. Único logro que no toca el léxico: `++` y `--` ya eran tokens de la Etapa 1.
Atributos Inicializados	`atrinit`	`<InicializacionDeAtributo>`.
Inicializacion de Arreglos E2	`arrinit`	El inicializador, anidable, sólo bajo **new**.
Genericidad Avanzada E2	`generico`	Genéricos anidados y la notación diamante.
Recuperación Modo Pánico	`modopanico`	No toca la gramática: es enteramente implementación.
Floats (Etapa 1)	`float`	`<TipoPrimitivo>` → **float** y `<Primitivo>` → **floatLiteral**.

Los otros tres —*Entrega Anticipada*, *Imbatibilidad* y *Well Done!*— también se persiguen, pero no se implementan por código: no dependen del compilador.

11 Verificación

Los conjuntos de Primeros se calcularon a mano y se contrastaron con una herramienta escrita para este trabajo, `herramientas/primeros.py`. La herramienta existe para **contrastar** la cuenta manual, no para reemplazarla: cuando las dos difirieron, se resolvió primero cuál de las dos estaba equivocada.

11.1 Condición LL(1)

```
$ python3 herramientas/primeros.py gramatica/minijava-base.g
--- 60 no terminales, 126 producciones, 51 terminales ---
NO ES LL(1): 16 conflicto(s)

$ python3 herramientas/primeros.py gramatica/minijava-ll1-logros.g
--- 111 no terminales, 228 producciones, 60 terminales ---
ES LL(1): toda produccion de todo no terminal tiene Primeros disjuntos.
```

11.2 Atomicidad de los logros

La condición de la sección 6.1 exige que la gramática sea LL(1) para *todo* subconjunto de logros, no sólo con todos prendidos. Cada producción lleva su marca y la herramienta recorre las $2^{10} = 1024$ configuraciones:

```
$ python3 herramientas/primeros.py gramatica/minijava-ll1-logros.g --combinatoria
10 logros -> 1024 configuraciones

LAS 1024 CONFIGURACIONES SON LL(1).
Cada logro se puede prender y apagar de forma independiente.
```

11.3 Ambigüedad frente a conflicto de predicción

Para distinguir los dos defectos de la sección 2.2 se comparan los rangos de longitud derivable: si no se solapan, los lenguajes son disjuntos y no puede haber ambigüedad.

```
$ python3 herramientas/primeros.py gramatica/minijava-base.g \
      --longitudes VarEncadenada MetodoEncadenado
<VarEncadenada>: entre 2 y 2 tokens
<MetodoEncadenado>: entre 4 y inf tokens

Los rangos NO se solapan: ninguna cadena puede estar en los dos
lenguajes, asi que <VarEncadenada> y <MetodoEncadenado> NO son ambiguos
entre si. Si comparten prefijo izquierdo es un conflicto de PREDICCION
(LL(1)), no de ambiguedad, y se arregla factorizando.
```

11.4 Registro de ambigüedades

La condición LL(1) que verifica el apartado anterior es la de la cátedra: disjunción de Primeros. El ítem «no es ambigua, o es arreglable» del checklist de Clase 4, slide 41, es otro, y se audita aparte. La herramienta recorre las 1024 configuraciones, junta toda ambigüedad que aparezca y **exige que cada una esté declarada con su regla de resolución**: si aparece una nueva, falla.

```
$ python3 herramientas/primeros.py gramatica/minijava-ll1-logros.g --ambiguedades
DIAGNOSTICO OFFLINE - no forma parte del metodo de implementacion.
El analizador sintactico decide SOLO con Primeros (Esquema Simple,
Clase 5). Los Siguientes se usan aca, y solo aca, para auditar el item
"no es ambigua o es arreglable" del checklist de Clase 4 slide 41.

Ambiguedades detectadas en las 1024 configuraciones: 3

  <DimensionesConTamanioOpcional>  ...  [declarada]
  <ElseOpcional>                   ...  [declarada]
  <RestoExpresion>                 ...  [declarada]

Todas las ambiguedades detectadas estan declaradas con su regla de resolucion.
```

Eso permite afirmar, con evidencia y no por revisión a ojo, que **éstas son todas las ambigüedades de la gramática** y que cada una tiene una resolución explícita.

11.5 Casos de aceptación y rechazo

La herramienta incluye un simulador del esquema de traducción — elige la producción cuyo cuerpo tiene al token actual en sus Primeros, y si ninguna lo tiene pero el no terminal es vaciable, no hace nada — que permite probar la gramática **antes** de escribir el analizador. Se verifican **87 casos**: por cada logro, la sintaxis que debe aceptar y la que debe rechazar cuando está apagado.

```
$ bash herramientas/casos-gramatica.sh
--- nucleo (sin ningun logro) ---
  ok    clase vacia
  ok    if/else anidado
  ok    encadenamiento
  ok    a = b = c NO es valido
--- MiniLambdas ---
  ok    x -> x            (1 param sin parentesis)
  ok    ( ) -> 1          (0 params)
  ok    ( a ) -> a        (1 param con parentesis)
  ok    ( a , b ) -> a    (2 params)
  ok    ( a ) + 1         (parentizada, NO lambda)
  ok    cuerpo con llaves NO se permite
  ok    parametro con tipo NO se permite
--- Genericidad Avanzada ---
  ok    List < List < Persona > > x ;  (anidado)
  ok    new List < > ( )               (diamante)
  ok    List < > x ;    diamante en posicion de TIPO
  ok    new List < > [ 3 ]  diamante con arreglo
...
TODOS LOS CASOS PASAN.
```

Los casos que fijan las decisiones difíciles de este informe son, entre otros: `a = b = c` se rechaza (no asociatividad de la asignación); `( a ) + 1` se acepta como expresión parentizada y `( a ) -> a` como lambda, decidiéndose con un solo token; `List<> x;` se rechaza y `new List<>()` se acepta.

11.6 Validación contra los casos de la cátedra

La verificación más fuerte no usa casos propios. La herramienta `aceptar_fuente.py` tokeniza un fuente **real** con el analizador léxico de la Etapa 1 y le pasa la secuencia de tokens al simulador, de modo que la gramática se puede probar contra los ocho casos entregados con los testers de esta etapa antes de que exista el analizador sintáctico.

```
$ python3 herramientas/aceptar_fuente.py \
      ../vendor/testersSintactico/resources/*/*.java
  ok    sintCorrecto01.java      acepta
  ok    sintCorrecto02.java      acepta
  ok    sintCorrecto03.java      acepta
  ok    sintCorrecto04.java      acepta
  ok    sintError01.java         rechaza  [Error:,|3] ==
        se esperaba ‘{‘ y se encontro ‘,‘
  ok    sintError02.java         rechaza  [Error:clasesita|3] ==
        se esperaba ‘idClase‘ y se encontro ‘idMetVar‘
  ok    sintError03.java         rechaza  [Error:hola|5] ==
        se esperaba ‘;‘ y se encontro ‘idMetVar‘
  ok    sintError04.java         rechaza  [Error:else|7] ==
```

```
          se esperaba '}' y se encontro 'else'

TODOS LOS CASOS PASAN.
```

Los cuatro casos válidos se aceptan y los cuatro inválidos se rechazan. Además —y esto es lo que importa para la corrección— el rechazo se produce **en el token exacto** que declara la expectativa de cada caso: el `==` indica que el código `[Error:lexema|línea]` derivado de la gramática coincide con el que la cátedra espera en los cuatro. La gramática no sólo reconoce el lenguaje correcto: falla donde tiene que fallar.

11.7 Tres reconocedores independientes de la misma gramática

El argumento de corrección más fuerte del trabajo no es que el analizador acepte una lista de casos, sino que **tres implementaciones escritas por separado coincidan**. Las tres reconocen la misma gramática y salen del mismo archivo, pero no comparten una línea de código:

El analizador de la entrega Java, descendente recursivo, Esquema Simple de Clase 5. Elige mirando los Primeros que `primeros_a_java.py` genera desde la gramática.

El simulador Python, con pila explícita, en `primeros.py`. Imita el mismo esquema pero sin recursión, y existía antes que el analizador.

El oráculo JavaCC generado por `gramatica_a_javacc.py`. JavaCC es un generador de parsers de terceros: se le da la gramática y escribe el parser. Con `LOOKAHEAD = 1` usa exactamente la misma técnica.

Decisión — qué prueba, y qué no prueba, que JavaCC no emita avisos

JavaCC avisa `Choice conflict involving two expansions` cuando dos alternativas de un mismo no terminal empiezan con el mismo token. Sobre la gramática final **no emite ninguno**, lo que es una confirmación independiente —de una herramienta que calculó los conjuntos por su cuenta sobre las 228 producciones— de la condición LL(1) de la cátedra: disjunción de Primeros.

No confirma la condición fuerte. Se comprobó con dos gramáticas de control: ante `<S>` → **a b** | **a c** JavaCC avisa; ante `<Opt>` → **b** | ε seguido de **b** —un conflicto Primeros–Siguientes— no avisa nada. Las tres ambigüedades de la sección 4.3 son de esa segunda clase, así que el silencio de JavaCC no dice nada sobre ellas y siguen auditándose con el registro propio.

Lo que sí lo hace útil como oráculo es cómo resuelve lo que no puede predecir: JavaCC toma **la primera alternativa que coincide**, que es exactamente la resolución voraz que aplica este analizador y la que la sección 4.3 declara para las tres. Por eso las tres implementaciones coinciden también en los casos ambiguos, y por eso el oráculo puede etiquetar el corpus: cuando dice `[Error:lexema|línea]`, ése es el error que el analizador tiene que dar.

11.8 Equivalencia de lenguajes con la gramática de la cátedra

Clase 4 exige adaptar la gramática *«sin cambiar el lenguaje que genera»*. Eliminar recursión a izquierda y factorizar lo preservan *en teoría*; `equivalencia_de_lenguajes.py` lo comprueba *en la práctica* sobre las dos gramáticas concretas. Deriva una cadena por

cada producción de cada gramática —dos por cada producción anidable, porque un solo uso no distingue `a = b` de `a = b = c`— y verifica que la otra la acepte. Como la gramática base no es LL(1), reconocerla necesita un algoritmo que tolere recursión a izquierda y ambigüedad: se usa un reconocedor de Earley.

```
$ python3 herramientas/equivalencia_de_lenguajes.py
minijava-base.g: 126/126 producciones ejercitadas, 0 inertes
minijava-ll1-logros.g (cero logros): 154/159 producciones ejercitadas, 5 inertes
    inerte    ExpresionSinIdClase -> ...
    ...

579 mutantes comparados en las dos gramaticas.
LAS DOS GRAMATICAS GENERAN EL MISMO LENGUAJE: toda produccion de cada una fue
ejercitada y la otra estuvo de acuerdo.
```

Las cinco producciones *inertes* son las de `<ExpresionSinIdClase>` y sus colas, que existen para compartirse entre dos logros y con cero logros no las referencia nadie. La cobertura por producción es lo que le da fuerza al chequeo: se validó contra dos controles negativos —uno que ensancha el lenguaje final permitiendo `a = b = c` y otro que lo angosta quitando `<Sentencia>` → `<Bloque>`— y los detecta a los dos.

11.9 El analizador cita exactamente la gramática

Cada método del analizador lleva arriba, en un comentario, las producciones que traduce. `auditar_analizador.py` compara esas citas contra la gramática y falla si una producción no la implementa nadie o si un comentario quedó viejo:

```
$ python3 herramientas/auditar_analizador.py
234 producciones en la gramatica, 234 citadas por el analizador.
EL ANALIZADOR CITA EXACTAMENTE LAS PRODUCCIONES DE LA GRAMATICA.
```

11.10 Resumen

	Gramática base	**Gramática final**
No terminales	60	111
Producciones	126	228
Terminales	51	60
Conflictos Primeros–Primeros	16	0
Ambigüedades sin declarar	2	0
Ambigüedades declaradas y resueltas	0	3

El crecimiento se reparte entre la corrección del núcleo y las nueve extensiones por logro. No queda ningún conflicto de predicción en ninguna de las 1024 configuraciones, y las tres ambigüedades que quedan están declaradas con su regla de resolución, que es lo que pide Clase 4, slide 49.

11.11 Tabla de Primeros

Conjunto de Primeros de cada producción de la gramática final, con todos los logros prendidos.

<Inicial> ::= <ListaClases> **eof**

Primeros = { **class**, **eof**, **interface** }

<ListaClases> ::= <Clase> <ListaClases>

Primeros = { **class** }

<ListaClases> ::= <Interfaz> <ListaClases>

Primeros = { **interface** }

<ListaClases> ::= ϵ

Primeros = { ϵ}

<Clase> ::= **class idClase** <GenericidadOpcional> <HerenciaOpcional> **{** <ListaMiembros> **}**

Primeros = { **class** }

<Interfaz> ::= **interface idClase** <GenericidadOpcional> <ExtensionOpcional> **{** <ListaMetodosInterfaz> **}**

Primeros = { **interface** }

<GenericidadOpcional> ::= **< idGen >**

Primeros = { < }

<GenericidadOpcional> ::= ϵ

Primeros = { ϵ}

<HerenciaOpcional> ::= **extends** <TipoReferencia>

Primeros = { **extends** }

<HerenciaOpcional> ::= **implements** <TipoReferencia>

Primeros = { **implements** }

<HerenciaOpcional> ::= ϵ

Primeros = { ϵ}

<ExtensionOpcional> ::= **extends** <TipoReferencia>

Primeros = { **extends** }

<ExtensionOpcional> ::= ϵ

Primeros = { ϵ}

<ListaMiembros> ::= <Miembro> <ListaMiembros>

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int**, **private**, **public**, **static**, **void** }

<ListaMiembros> ::= ϵ

Primeros = { ϵ}

<ListaMetodosInterfaz> ::= <MetodoInterfaz> <ListaMetodosInterfaz>

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int**, **public**, **void** }

<ListaMetodosInterfaz> ::= ϵ

Primeros = { ϵ}

<Miembro> ::= <VisibilidadOpcional> <MiembroSinVisibilidad>

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int**, **private**, **public**, **static**, **void** }

<Miembro> ::= <MiembroSinVisibilidad>

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int**, **static**, **void** }

<Miembro> ::= <Constructor>

Primeros = { **public** }

<VisibilidadOpcional> ::= **public**

Primeros = { **public** }

<VisibilidadOpcional> ::= **private**

Primeros = { **private** }

<VisibilidadOpcional> ::= ϵ

Primeros = { ϵ}

<Constructor> ::= **public idClase** <CuerpoDeMetodo>

Primeros = { **public** }

<MiembroSinVisibilidad> ::= **static** <TipoMetodo> <Metodo>

Primeros = { **static** }

<MiembroSinVisibilidad> ::= **void** <Metodo>

Primeros = { **void** }

<MiembroSinVisibilidad> ::= <TipoPrimitivo> <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

Primeros = { **boolean**, **char**, **float**, **int** }

<MiembroSinVisibilidad> ::= **idGen** <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

Primeros = { **idGen** }

<MiembroSinVisibilidad> ::= **idClase** <MiembroDesdeIdClase>

Primeros = { **idClase** }

<MiembroDesdeIdClase> ::= <CuerpoDeMetodo>

Primeros = { **(** }

<MiembroDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> **idMetVar** <AtributoOMetodo>

Primeros = { $<$, **[**, **idMetVar** }

<AtributoOMetodo> ::= <InicializacionDeAtributo> **;**

Primeros = { **;**, **=** }

<AtributoOMetodo> ::= <CuerpoDeMetodo>

Primeros = { **(** }

<InicializacionDeAtributo> ::= **=** <ExpresionCondicionalOLambda>

Primeros = { **=** }

<InicializacionDeAtributo> ::= ϵ

Primeros = { ϵ}

<Metodo> ::= **idMetVar** <CuerpoDeMetodo>

Primeros = { **idMetVar** }

<CuerpoDeMetodo> ::= <ArgsFormales> <Bloque>

Primeros = { **(** }

<MetodoInterfaz> ::= <PublicOpcional> <TipoMetodo> **idMetVar** <ArgsFormales> **;**

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int**, **public**, **void** }

`<PublicOpcional>` ::= **public**

Primeros = { **public** }

`<PublicOpcional>` ::= ϵ

Primeros = { ϵ}

`<TipoMetodo>` ::= `<Tipo>`

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int** }

`<TipoMetodo>` ::= **void**

Primeros = { **void** }

`<Tipo>` ::= `<TipoBase>` `<DimensionesOpcionales>`

Primeros = { **boolean**, **char**, **float**, **idClase**, **idGen**, **int** }

`<TipoBase>` ::= `<TipoPrimitivo>`

Primeros = { **boolean**, **char**, **float**, **int** }

`<TipoBase>` ::= `<TipoReferencia>`

Primeros = { **idClase** }

`<TipoBase>` ::= **idGen**

Primeros = { **idGen** }

`<DimensionesOpcionales>` ::= **[ ]** `<DimensionesOpcionales>`

Primeros = { **[** }

`<DimensionesOpcionales>` ::= ϵ

Primeros = { ϵ}

`<TipoReferencia>` ::= **idClase** `<TipoGenericoOpcional>`

Primeros = { **idClase** }

`<TipoPrimitivo>` ::= **boolean**

Primeros = { **boolean** }

`<TipoPrimitivo>` ::= **char**

Primeros = { **char** }

`<TipoPrimitivo>` ::= **int**

Primeros = { **int** }

`<TipoPrimitivo>` ::= **float**

Primeros = { **float** }

`<TipoGenericoOpcional>` ::= < `<ArgumentoDeTipo>` >

Primeros = { < }

`<TipoGenericoOpcional>` ::= ϵ

Primeros = { ϵ}

`<ArgumentoDeTipo>` ::= **idGen**

Primeros = { **idGen** }

`<ArgumentoDeTipo>` ::= **idClase** `<TipoGenericoOpcional>`

Primeros = { **idClase** }

`<ArgumentoDeTipo>` ::= **idClase**

Primeros = { **idClase** }

<ArgsFormales> ::= (<ListaArgsFormalesOpcional>)

Primeros = { (}

<ListaArgsFormalesOpcional> ::= <ListaArgsFormales>

Primeros = { boolean, char, float, idClase, idGen, int }

<ListaArgsFormalesOpcional> ::= ϵ

Primeros = { ϵ}

<ListaArgsFormales> ::= <ArgFormal> <RestoListaArgsFormales>

Primeros = { boolean, char, float, idClase, idGen, int }

<RestoListaArgsFormales> ::= , <ArgFormal> <RestoListaArgsFormales>

Primeros = { , }

<RestoListaArgsFormales> ::= ϵ

Primeros = { ϵ}

<ArgFormal> ::= <Tipo> idMetVar

Primeros = { boolean, char, float, idClase, idGen, int }

<Bloque> ::= { <ListaSentencias> }

Primeros = { { }

<ListaSentencias> ::= <Sentencia> <ListaSentencias>

Primeros = { !, (, +, -, ;, boolean, char, charLiteral, false, float, floatLiteral, for, idClase, idGen, idMetVar, if, int, intLiteral, new, null, return, stringLiteral, this, true, var, while, { }

<ListaSentencias> ::= ϵ

Primeros = { ϵ}

<Sentencia> ::= ;

Primeros = { ; }

<Sentencia> ::= <VarLocal> ;

Primeros = { var }

<Sentencia> ::= <Return> ;

Primeros = { return }

<Sentencia> ::= <If>

Primeros = { if }

<Sentencia> ::= <While>

Primeros = { while }

<Sentencia> ::= <Bloque>

Primeros = { { }

<Sentencia> ::= <For>

Primeros = { for }

<Sentencia> ::= <SentenciaExpresion> ;

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<Sentencia> ::= <ExpresionSinIdClase> ;

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<Sentencia> ::= <TipoPrimitivo> <DimensionesOpcionales> <ListaDeclaradores> ;

Primeros = { boolean, char, float, int }

<Sentencia> ::= idGen <DimensionesOpcionales> <ListaDeclaradores> ;

Primeros = { idGen }

<Sentencia> ::= idClase <SentenciaDesdeIdClase>

Primeros = { idClase }

<SentenciaExpresion> ::= <ExpresionSinLambda>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<SentenciaDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> <ListaDeclaradores> ;

Primeros = { <, [, idMetVar }

<SentenciaDesdeIdClase> ::= . idMetVar <ArgsActuales> <ColaDeExpresionDesdePrimario> ;

Primeros = { . }

<VarLocal> ::= var idMetVar = <ExpresionCondicionalOLambda>

Primeros = { var }

<ListaDeclaradores> ::= idMetVar <InicializacionDeDeclarador> <RestoListaDeclaradores>

Primeros = { idMetVar }

<RestoListaDeclaradores> ::= , idMetVar <InicializacionDeDeclarador> <RestoListaDeclaradores>

Primeros = { , }

<RestoListaDeclaradores> ::= ϵ

Primeros = { ϵ}

<InicializacionDeDeclarador> ::= = <ExpresionCondicionalOLambda>

Primeros = { = }

<InicializacionDeDeclarador> ::= ϵ

Primeros = { ϵ}

<Return> ::= return <ExpresionOpcional>

Primeros = { return }

<ExpresionOpcional> ::= <Expresion>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<ExpresionOpcional> ::= ϵ

Primeros = { ϵ}

<If> ::= if (<Expresion>) <Sentencia> <ElseOpcional>

Primeros = { if }

<ElseOpcional> ::= else <Sentencia>

Primeros = { else }

<ElseOpcional> ::= ϵ

Primeros = { ϵ}

<While> ::= while (<Expresion>) <Sentencia>

Primeros = { while }

<For> ::= for (<EncabezadoFor>) <Sentencia>

Primeros = { for }

<EncabezadoFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>

Primeros = { ; }

<EncabezadoFor> ::= var idMetVar <EncabezadoForTrasVar>

Primeros = { var }

<EncabezadoFor> ::= <TipoPrimitivo> <DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador>

Primeros = { boolean, char, float, int }

<EncabezadoFor> ::= idGen <DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador>

Primeros = { idGen }

<EncabezadoFor> ::= idClase <EncabezadoForDesdeIdClase>

Primeros = { idClase }

<EncabezadoFor> ::= <ExpresionSinIdClase> <ClausulasFinalesFor>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<EncabezadoForDesdeIdClase> ::= <TipoGenericoOpcional> <DimensionesOpcionales> idMetVar <EncabezadoForTrasDeclarador>

Primeros = { <, [, idMetVar }

<EncabezadoForDesdeIdClase> ::= . idMetVar <ArgsActuales> <ColaDeExpresionDesdePrimario> <ClausulasFinalesFor>

Primeros = { . }

<EncabezadoForTrasVar> ::= : <Expresion>

Primeros = { : }

<EncabezadoForTrasVar> ::= = <ExpresionCondicionalOLambda> <ClausulasFinalesFor>

Primeros = { = }

<EncabezadoForTrasDeclarador> ::= : <Expresion>

Primeros = { : }

<EncabezadoForTrasDeclarador> ::= <InicializacionDeDeclaradorFor> <ClausulasFinalesFor>

Primeros = { ;, = }

<InicializacionDeDeclaradorFor> ::= = <ExpresionCondicionalOLambda>

Primeros = { = }

<InicializacionDeDeclaradorFor> ::= ϵ

Primeros = { ϵ}

<ClausulasFinalesFor> ::= ; <ExpresionOpcional> ; <ExpresionOpcional>

Primeros = { ; }

<Expresion> ::= <ExpresionCondicionalOLambda> <RestoExpresion>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<ExpresionSinLambda> ::= <ExpresionCondicional> <RestoExpresion>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<RestoExpresion> ::= <OperadorAsignacion> <ExpresionCondicionalOLambda>

Primeros = { = }

<RestoExpresion> ::= ϵ

Primeros = { ϵ}

<OperadorAsignacion> ::= =

Primeros = { = }

<ExpresionCondicionalOLambda> ::= <OperadorUnario> <Operando> <ColaTrasOperandoSinAsignacion>

Primeros = { !, +, - }

<ExpresionCondicionalOLambda> ::= <Primitivo> <ColaTrasOperandoSinAsignacion>

Primeros = { charLiteral, false, floatLiteral, intLiteral, null, true }

<ExpresionCondicionalOLambda> ::= <ExpresionDesdePrimarioOLambda>

Primeros = { (, idClase, idMetVar, new, stringLiteral, this }

<ExpresionDesdePrimarioOLambda> ::= idClase . idMetVar <ArgsActuales> <ColaDesdePrimarioSinAsignacion>

Primeros = { idClase }

<ExpresionDesdePrimarioOLambda> ::= this <ColaDesdePrimarioSinAsignacion>

Primeros = { this }

<ExpresionDesdePrimarioOLambda> ::= stringLiteral <ColaDesdePrimarioSinAsignacion>

Primeros = { stringLiteral }

<ExpresionDesdePrimarioOLambda> ::= <Creacion> <ColaDesdePrimarioSinAsignacion>

Primeros = { new }

<ExpresionDesdePrimarioOLambda> ::= idMetVar <AccesoLlamadaOLambda>

Primeros = { idMetVar }

<ExpresionDesdePrimarioOLambda> ::= <ParentizadaOLambda>

Primeros = { (}

<ExpresionCondicional> ::= <ExpresionCompuesta> <RestoCondicional>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<RestoCondicional> ::= ? <Expresion> : <ExpresionCondicionalOLambda>

Primeros = { ? }

<RestoCondicional> ::= ϵ

Primeros = { ϵ}

<ExpresionCompuesta> ::= <ExpresionBasica> <RestoExpresionCompuesta>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<RestoExpresionCompuesta> ::= <OperadorBinario> <ExpresionBasica>

<RestoExpresionCompuesta>
Primeros = { !=, %, &&, *, +, -, /, <, <=, ==, >, >=, || }

<RestoExpresionCompuesta> ::= ϵ
Primeros = { ϵ}

<OperadorBinario> ::= ||
Primeros = { || }

<OperadorBinario> ::= &&
Primeros = { && }

<OperadorBinario> ::= ==
Primeros = { == }

<OperadorBinario> ::= !=
Primeros = { != }

<OperadorBinario> ::= <
Primeros = { < }

<OperadorBinario> ::= >
Primeros = { > }

<OperadorBinario> ::= <=
Primeros = { <= }

<OperadorBinario> ::= >=
Primeros = { >= }

<OperadorBinario> ::= +
Primeros = { + }

<OperadorBinario> ::= -
Primeros = { - }

<OperadorBinario> ::= *
Primeros = { * }

<OperadorBinario> ::= /
Primeros = { / }

<OperadorBinario> ::= %
Primeros = { % }

<ExpresionBasica> ::= <OperadorUnario> <Operando>
Primeros = { !, +, - }

<ExpresionBasica> ::= <Operando>
Primeros = { (, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<OperadorUnario> ::= +
Primeros = { + }

<OperadorUnario> ::= -
Primeros = { - }

<OperadorUnario> ::= !
Primeros = { ! }

<Operando> ::= <Primitivo>

Primeros = { charLiteral, false, floatLiteral, intLiteral, null, true }

<Operando> ::= <Referencia> <PosfijoOpcional>

Primeros = { (, idClase, idMetVar, new, stringLiteral, this }

<PosfijoOpcional> ::= ++

Primeros = { ++ }

<PosfijoOpcional> ::= --

Primeros = { -- }

<PosfijoOpcional> ::= ϵ

Primeros = { ϵ}

<Primitivo> ::= true

Primeros = { true }

<Primitivo> ::= false

Primeros = { false }

<Primitivo> ::= intLiteral

Primeros = { intLiteral }

<Primitivo> ::= charLiteral

Primeros = { charLiteral }

<Primitivo> ::= null

Primeros = { null }

<Primitivo> ::= floatLiteral

Primeros = { floatLiteral }

<ExpresionSinIdClase> ::= <OperadorUnario> <Operando> <ColaTrasOperando>

Primeros = { !, +, - }

<ExpresionSinIdClase> ::= <Primitivo> <ColaTrasOperando>

Primeros = { charLiteral, false, floatLiteral, intLiteral, null, true }

<ExpresionSinIdClase> ::= <PrimarioSinIdClase> <ColaDeExpresionDesdePrimario>

Primeros = { (, idMetVar, new, stringLiteral, this }

<ColaTrasOperando> ::= <ColaTrasOperandoSinAsignacion> <RestoExpresion>

Primeros = { !=, %, &&, *, +, -, /, <, <=, =, ==, >, >=, ?, ||, ϵ}

<ColaTrasOperandoSinAsignacion> ::= <RestoExpresionCompuesta>
<RestoCondicional>

Primeros = { !=, %, &&, *, +, -, /, <, <=, ==, >, >=, ?, ||, ϵ}

<ColaDeExpresionDesdePrimario> ::= <RestoReferencia> <PosfijoOpcional>
<ColaTrasOperando>

Primeros = { !=, %, &&, *, +, ++, -, --, ., /, <, <=, =, ==, >, >=, ?, [, ||, ϵ}

<ColaDesdePrimarioSinAsignacion> ::= <RestoReferencia> <PosfijoOpcional>
<ColaTrasOperandoSinAsignacion>

Primeros = { !=, %, &&, *, +, ++, -, --, ., /, <, <=, ==, >, >=, ?, [, ||, ϵ}

<Referencia> ::= <Primario> <RestoReferencia>

Primeros = { (, idClase, idMetVar, new, stringLiteral, this }

<RestoReferencia> ::= <Encadenado> <RestoReferencia>

Primeros = { . }

<RestoReferencia> ::= <AccesoArreglo> <RestoReferencia>

Primeros = { [}

<RestoReferencia> ::= ϵ

Primeros = { ϵ}

<Encadenado> ::= . idMetVar <EncadenadoAtributoOMetodo>

Primeros = { . }

<EncadenadoAtributoOMetodo> ::= <ArgsActuales>

Primeros = { (}

<EncadenadoAtributoOMetodo> ::= ϵ

Primeros = { ϵ}

<AccesoArreglo> ::= [<Expresion>]

Primeros = { [}

<Primario> ::= <LlamadaMetodoEstatico>

Primeros = { idClase }

<Primario> ::= <PrimarioSinIdClase>

Primeros = { (, idMetVar, new, stringLiteral, this }

<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>

Primeros = { idClase }

<PrimarioSinIdClase> ::= this

Primeros = { this }

<PrimarioSinIdClase> ::= stringLiteral

Primeros = { stringLiteral }

<PrimarioSinIdClase> ::= <Parentizada>

Primeros = { (}

<PrimarioSinIdClase> ::= <AccesoVarOLlamada>

Primeros = { idMetVar }

<PrimarioSinIdClase> ::= <Creacion>

Primeros = { new }

<Parentizada> ::= (<ExpresionSinLambda>)

Primeros = { (}

<AccesoVarOLlamada> ::= idMetVar <AccesoLlamada>

Primeros = { idMetVar }

<AccesoLlamada> ::= <ArgsActuales>

Primeros = { (}

<AccesoLlamada> ::= ϵ

Primeros = { ϵ}

<AccesoLlamadaOLambda> ::= -> <CuerpoLambda>

Primeros = { -> }

<AccesoLlamadaOLambda> ::= <AccesoLlamada> <ColaDesdePrimarioSinAsignacion>

Primeros = { !=, %, &&, (, *, +, ++, -, --, ., /, <, <=, ==, >, >=, ?, [, ||, ϵ}

<ParentizadaOLambda> ::= (<ContenidoDeParentesis>

Primeros = { (}

<ContenidoDeParentesis> ::=) -> <CuerpoLambda>

Primeros = {) }

<ContenidoDeParentesis> ::= <Expresion> <RestoDeParentesis>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<RestoDeParentesis> ::=) <FinDeParentizadaOLambda>

Primeros = {) }

<RestoDeParentesis> ::= , <ListaParametrosLambda>) -> <CuerpoLambda>

Primeros = { , }

<FinDeParentizadaOLambda> ::= -> <CuerpoLambda>

Primeros = { -> }

<FinDeParentizadaOLambda> ::= <ColaDesdePrimarioSinAsignacion>

Primeros = { !=, %, &&, *, +, ++, -, --, ., /, <, <=, ==, >, >=, ?, [, ||, ϵ}

<ListaParametrosLambda> ::= idMetVar <RestoListaParametrosLambda>

Primeros = { idMetVar }

<RestoListaParametrosLambda> ::= , idMetVar <RestoListaParametrosLambda>

Primeros = { , }

<RestoListaParametrosLambda> ::= ϵ

Primeros = { ϵ}

<CuerpoLambda> ::= <Expresion>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<Creacion> ::= new <ArregloOInstancia>

Primeros = { new }

<ArregloOInstancia> ::= <TipoPrimitivo> <CreacionDeArreglo>

Primeros = { boolean, char, float, int }

<ArregloOInstancia> ::= idGen <CreacionDeArreglo>

Primeros = { idGen }

<ArregloOInstancia> ::= idClase <CreacionDesdeIdClase>

Primeros = { idClase }

<CreacionDesdeIdClase> ::= < <GenericoDeCreacion>

Primeros = { < }

<CreacionDesdeIdClase> ::= <CreacionDeArreglo>

Primeros = { [}

<CreacionDesdeIdClase> ::= <ArgsActuales>

Primeros = { (}

<GenericoDeCreacion> ::= > <ArgsActuales>

Primeros = { > }

<GenericoDeCreacion> ::= <ArgumentoDeTipo> > <ArregloOInstanciaDeClase>

Primeros = { idClase, idGen }

<ArregloOInstanciaDeClase> ::= <CreacionDeArreglo>

Primeros = { [}

<ArregloOInstanciaDeClase> ::= <ArgsActuales>

Primeros = { (}

<CreacionDeArreglo> ::= [<RestoCreacionDeArreglo>

Primeros = { [}

<RestoCreacionDeArreglo> ::= <Expresion>] <DimensionesConTamanioOpcional>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<RestoCreacionDeArreglo> ::=] <DimensionesVaciasOpcional>
<InicializadorDeArreglo>

Primeros = {] }

<DimensionesVaciasOpcional> ::= [ ] <DimensionesVaciasOpcional>

Primeros = { [}

<DimensionesVaciasOpcional> ::= ϵ

Primeros = { ϵ}

<InicializadorDeArreglo> ::= { <ListaElementosOpcional> }

Primeros = { { }

<ListaElementosOpcional> ::= <ListaElementos>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true, { }

<ListaElementosOpcional> ::= ϵ

Primeros = { ϵ}

<ListaElementos> ::= <ElementoInicializador> <RestoListaElementos>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true, { }

<RestoListaElementos> ::= , <ElementoInicializador> <RestoListaElementos>

Primeros = { , }

<RestoListaElementos> ::= ϵ

Primeros = { ϵ}

<ElementoInicializador> ::= <Expresion>

Primeros = { !, (, +, -, charLiteral, false, floatLiteral, idClase, idMetVar, intLiteral, new, null, stringLiteral, this, true }

<ElementoInicializador> ::= <InicializadorDeArreglo>

Primeros = { { }

<DimensionesConTamanio> ::= [<Expresion>] <DimensionesConTamanioOpcional>

Primeros = { [}

<DimensionesConTamanioOpcional> ::= <DimensionesConTamanio>

Primeros = { [}

<DimensionesConTamanioOpcional> ::= ϵ

Primeros = { ϵ}

`<ArgsActuales>` ::= **(** `<ListaExpsOpcional>` **)**

Primeros = { **(** }

`<ListaExpsOpcional>` ::= `<ListaExps>`

Primeros = { **!**, **(**, **+**, **-**, **charLiteral**, **false**, **floatLiteral**, **idClase**, **idMetVar**, **intLiteral**, **new**, **null**, **stringLiteral**, **this**, **true** }

`<ListaExpsOpcional>` ::= ϵ

Primeros = { ϵ}

`<ListaExps>` ::= `<Expresion>` `<RestoListaExps>`

Primeros = { **!**, **(**, **+**, **-**, **charLiteral**, **false**, **floatLiteral**, **idClase**, **idMetVar**, **intLiteral**, **new**, **null**, **stringLiteral**, **this**, **true** }

`<RestoListaExps>` ::= **,** `<Expresion>` `<RestoListaExps>`

Primeros = { **,** }

`<RestoListaExps>` ::= ϵ

Primeros = { ϵ}

12 Decisiones tomadas con criterio propio

Buena parte del trabajo de esta etapa no fue aplicar transformaciones mecánicas sino *decidir* dónde la especificación admitía más de una lectura. Esta sección junta todas esas decisiones en un solo lugar, cada una con la regla que se aplicó y con la fuente que la respalda.

El criterio de desempate, cuando lo hubo, es el que fija la cátedra: *«todo lo que NO esté especificado para MiniJava tiene comportamiento por defecto igual al de Java»*.

12.1 Ambigüedades: qué árbol se elige

Las tres ambigüedades del registro de la sección 11 se resuelven todas por regla de implementación —la vía que autoriza Clase 4, slide 49— y todas de la misma forma: **voraz**, consumiendo todo lo que se pueda:

Caso	Decisión
dangling else	El **else** liga al **if** más cercano. En el analizador es el orden de las ramas: se toma **else** apenas se lo ve. Coincide con Java.
`new int[3][4]`	Todas las dimensiones pertenecen a la creación; no es un acceso al arreglo recién creado. Coincide con Java, que lo lee como una creación 2-D. Heredada de la gramática de la cátedra.
Cuerpo de la lambda	`x -> a = b` es `x -> (a = b)`. La introduce este trabajo, y afecta a un solo no terminal: `<RestoExpresion>`, la única cola externa que todavía compite con la del cuerpo una vez que la lambda vive en el nivel más alto de la jerarquía (sección 4.3). Coincide con Java.

Además, `<Asignacion>` y `<Llamada>` tienen cuerpos idénticos en la gramática original y por lo tanto toda sentencia-expresión tiene dos árboles. Se **colapsan en una sola**

producción: distinguir una asignación de una llamada exige saber a qué se refiere el nombre de la izquierda, o sea una tabla de símbolos, y eso es trabajo del analizador semántico.

12.2 Dónde la especificación admitía más de una lectura

Asociatividad del ternario. A derecha: `a ? b : c ? d : e` es `a ? b : (c ? d : e)`. Es lo que hace Java.

Los dos operandos del ternario. El del medio es una `<Expresion>` completa y el de la derecha una `<ExpresionCondicional>`, tal cual el JLS 15.25. En consecuencia `a ? b = 1 : c` es válido y `a ? b : c = d` no lo es, que es exactamente lo que acepta y rechaza `javac`.

La asignación no encadena. Todo lado derecho de un **=** —el de la asignación y el de los cinco inicializadores— es una `<ExpresionCondicional>` y no una `<Expresion>`. **No es una decisión libre**: la regla original pone `<ExpresionCompuesta>` ahí, o sea que `a = b = c` nunca fue válido en MiniJava. Se preservó, y por eso tampoco vale dentro del operando del medio del ternario.

Genéricos anidados. `List<List<Persona>>` cierra con **dos > separados**: el analizador léxico de MiniJava no tiene un token >>, así que el problema clásico de Java no se presenta.

Visibilidad en métodos de interfaz. Se admite **public** y **no private**. Verificado contra `javac`: un método de interfaz **private** exige cuerpo (*missing method body, or declare abstract*), y en MiniJava los métodos de interfaz terminan en **;** y ningún logro les agrega cuerpo, así que un **private** ahí sería siempre la forma que Java rechaza.

Orden de los modificadores. `public static void f(){}` se acepta y `static public void f(){}` se rechaza. Es fiel a la gramática original, donde la visibilidad precede al modificador; Java admitiría los dos órdenes, pero acá la especificación *sí* dice algo y gana sobre la regla de desempate.

Inicializador de arreglos. Es **anidable**: `new int[][]{{1,2},{3,4}}` se acepta. El logro dice «como en Java» y sólo restringe *dónde* puede aparecer —dentro de un **new**— no cuántas dimensiones admite.

Notación diamante. Sólo al instanciar una clase genérica. `new C<>()` se acepta; `new C<>[3]` y `List<> x;` se rechazan. La restricción queda impuesta por la *gramática* —el diamante sólo aparece en la rama que va a `<ArgsActuales>`— y no delegada al semántico.

El declarador tipado del for. `for (int i = 0; ; )` funciona con el logro *Fors!* aunque el de variables locales clásicas esté apagado, porque el logro dice «**for** similar a como es en Java» y en Java el **for** clásico declara su propia variable. Por eso el encabezado tiene su propio `<InicializacionDeDeclaradorFor>` en vez de reusar el de las variables locales. La misma declaración *como sentencia* sigue necesitando el otro logro: los dos son independientes.

`float` en la gramática. **float** es token del léxico desde la Etapa 1 pero no figura en las *Reglas de Sintaxis*. Sin agregarlo como `<TipoPrimitivo>` y como `<Primitivo>`, el logro de la etapa anterior quedaba inutilizable en ésta.

El lexema del token EOF. Es **$**. El material de la cátedra se contradice: Pautas slide 26 y *Aspectos Léxicos* §2.5 dicen **$**, y la *Guía de uso de los testers — Etapa 2*, §5, dice que es vacío y que el código queda `[Error:|N]`. Como los dos formatos son incompatibles bajo la comparación por contenido que hace el tester, se consultó y se confirmó **$**.

12.3 Qué verifica el analizador y qué queda para el semántico

El logro *MiniLambdas E2* prohíbe usar **this** y capturar variables dentro de una lambda. Nada de eso es expresable en una gramática libre de contexto: no hay forma de escribir «**this** es válido, salvo debajo de un cuerpo de lambda». El criterio para decidir en qué etapa se verifica cada una fue: **si alcanza con información que el analizador acaba de leer, la verifica el analizador; si hace falta resolver un nombre contra su declaración, es del semántico.**

Restricción	Etapa	Por qué
this explícito `x -> this.c`	2	Pura forma: alcanza un indicador de «estoy dentro de una lambda», que se prende al entrar al cuerpo y se restaura al salir.
Captura `x -> x + n`	2	El cuerpo de una lambda es *una sola expresión*, así que adentro no se declara nada: los únicos nombres legales son sus propios parámetros, que el analizador acaba de leer.
Parámetro que no es un identificador pelado, o repetido `(a + 1) -> a`	2	Hasta ver la `->` el contenido entre paréntesis es una `<Expresion>` en los dos casos. El analizador la lee y recuerda si consumió exactamente un **idMetVar**.
Llamada sin punto `x -> ayuda(x)`	**3**	Es `this.ayuda(x)` *sólo si* **ayuda** es un método de instancia. Si es **static** de la misma clase no hay ningún **this** y Java lo acepta. Distinguirlos exige la tabla de símbolos, así que rechazarlo acá sería rechazar un programa válido.

Todo lo demás que dependa de resolver un nombre —que exista, que el tipo cierre, que la aridad coincida— es del semántico y no se toca en esta etapa.

12.4 Decisiones de implementación

Insertar el no terminal, no fusionar. Al factorizar, cuando había que elegir entre fundir dos contextos en un mismo no terminal o insertar uno nuevo por contexto, se insertó. Clase 4, slide 40, avisa que factorizar de más le complica el trabajo al semántico; el no terminal insertado es justamente el punto de decisión que la Etapa 3 va a necesitar.

La recursión a derecha se traduce con un bucle. Clase 5, slide 59, muestra el método de `<E'>` llamándose a sí mismo. Acá se traduce con un `while`: deriva exactamente la misma cadena y no hace depender de la pila de la JVM una cadena larga de

operadores o una lista larga de argumentos. Es la única libertad que se toma respecto del esquema.

Modo Pánico: dónde se recupera. En tres lugares y sólo tres —las tres listas de la gramática que repiten un elemento: clases, miembros y sentencias—, así el manejo del error no se desparrama por las diez secciones del analizador.

Modo Pánico: con qué se sincroniza. Con los tokens de Pautas slide 40 (`;`, `}` y la llave que abre) **más los Primeros del elemento de la lista**. Los Primeros hacen falta: sin ellos, un `int x` al que le falta el `;` se lleva puesto el método que viene después, porque el análisis falla justo en el `void` con el que empieza el miembro siguiente y correcto. Las llaves se cuentan mientras se descarta, así el cuerpo de un método con la firma rota se saltea entero en vez de leerse como si cada sentencia fuera otro miembro.

Modo Pánico: el formato del reporte. Se repite el reporte completo de Pautas slide 25 —mensaje, renglón en blanco, código— por cada error. Es el mismo formato con el que la Etapa 1 reporta varios errores léxicos con *Multi-detección*.

Modo Pánico: el primer error no cambia. El logro agrega errores al final de la lista, nunca cambia el primero. Es lo que permite que un mismo caso del corpus valga con el logro prendido y apagado, y hay un test que lo fija sobre todo el corpus.

13 Herramientas y uso de inteligencia artificial

13.1 Herramientas desarrolladas

Para este informe se escribieron dos utilidades, que no forman parte del compilador y viven en `compilador/herramientas/`:

`primeros.py` Calcula símbolos vaciables y conjuntos de Primeros por punto fijo, verifica la condición LL(1), recorre todas las combinaciones de logros, compara rangos de longitud derivable para distinguir ambigüedad de conflicto de predicción, y simula el esquema de traducción sobre una secuencia de tokens.

`gramatica_a_latex.py` Genera el listado de producciones y la tabla de Primeros de este informe desde el mismo archivo de gramática que verifica la herramienta anterior.

La gramática se mantiene en texto plano en `compilador/gramatica/`, en tres versiones: la base transcripta del documento de la cátedra, el núcleo corregido y la final con los logros.

13.2 Uso de inteligencia artificial

Este trabajo se desarrolló con asistencia de un modelo de lenguaje (Claude, de Anthropic), en línea con la política de la cátedra que acepta y fomenta su uso. Se deja constancia del reparto concreto:

- El **análisis de la gramática**, las transformaciones y las decisiones de diseño se elaboraron en diálogo con la herramienta, y cada resultado se verificó de manera independiente con `primeros.py` y con los 87 casos de aceptación y rechazo. Ninguna afirmación de este informe descansa únicamente en la salida del modelo.
- Las **herramientas** y la redacción del informe se escribieron con su asistencia.

- La **verificación** es mecánica y reproducible con los comandos de la sección 11, de modo que cualquier error es localizable en un paso concreto y no depende de confiar en el proceso.